# 창의융합종합설계 최종 결과보고서

<table>
<tr><td>팀 명칭</td><td colspan="4">SmartParcel</td></tr>
<tr><td rowspan="3">책 임 자</td><td>성 명</td><td colspan="3"></td></tr>
<tr><td>소 속</td><td colspan="3"></td></tr>
<tr><td>학 년</td><td></td><td>학 번</td><td></td></tr>
<tr><td>작품명칭</td><td colspan="4">AI-하드웨어 융합 소형 자동 분류 시스템</td></tr>
<tr><td>개발기간</td><td colspan="4">2025년 09월 03일 ~ 2025년 12월 12일</td></tr>
<tr><td rowspan="4">참여학생</td><td>학번</td><td>이름</td><td colspan="2">전공</td></tr>
<tr><td></td><td>김나영</td><td colspan="2">컴퓨터공학부 소프트웨어전공</td></tr>
<tr><td></td><td>장세연</td><td colspan="2">컴퓨터공학부 소프트웨어전공</td></tr>
<tr><td></td><td>박진호</td><td colspan="2">컴퓨터공학부 소프트웨어전공</td></tr>
</table>

본인은 소프트웨어공학심화프로그램 종합설계에 대한 최종결과보고서 및 작품을 첨부와 같이 제출합니다.

별첨: 아래 내용을 포함한 CD 혹은 USB 메모리

- 제안서, 결과 보고서
- 수업 시간 발표 자료
- 모든 회의록
- 개발 작품에 대한 관련 자료(동영상, 소스 파일등)

2025년 12월 12일

책임자 (인)

**소프트웨어공학심화프로그램 PD 귀하**

# < 목 차 >


1. 개요 ····· 1
1.1 문제 정의 ····· 2
1.2 목표 ····· 3

2. 요구사항 및 기능명세 ····· 4
2.1 요구사항 ····· 4
2.2 Use Case Diagram(유즈케이스 다이어그램) ····· 6
2.3 Use Case Descriptions(유즈케이스 명세) ····· 10

3. 시스템 구조도 ····· 40

4. 시스템 설계도 ····· 41

5. H/W 또는 시스템 구성 ····· 42
5.1 AI 비전 ····· 42
5.2 임베디드 구성 ····· 42
5.3 웹•모바일 대시보드 ····· 42

6. 구현 기술 ····· 44
6.1 기술 스택 ····· 44
6.2 구현 기술 ····· 44
6.3 성능평가 ····· 46

7. 주요 자료 구조 또는 DB 구조 ····· 54
7.1 데이터베이스 설계 ····· 54

8. 주요 함수 명세 ····· 55

9. 설치 매뉴얼 및 사용자 매뉴얼 ····· 84
9.1 설치 매뉴얼(Installation Manual) ····· 84

9.2 사용자 매뉴얼(User Manual) ····· 87

10. 역할 분담 ····· 92

11. 비고 ····· 93

12. 회의록 ····· 97

13. 팀원별 포트폴리오 ····· 105

# 1. 개요

군수·구호·재난 대응 등 소규모·임시적 현장에서는 다양한 물자를 즉시 분류하고 배치해야 하는 상황이 반복된다. 그러나 실제 현장은 제한된 공간, 불안정한 작업 환경, 인력 중심의 업무 구조가 중첩돼 있어 기존 자동 분류 설비의 도입이 현실적으로 어렵다. 특히 RFID·바코드 기반의 상용화된 자동화 장비는 고가이며 설치 제약이 많아, 설비를 갖춘 곳에서만 사용 가능한 폐쇄형 자동화라는 한계를 가진다.

이에 반해, 본 프로젝트는 어디서든 빠르게 설치·운용 가능하며, 현장에서 즉시 분류 기준을 바꿀 수 있는 ‘Physical AI 기반 지능형 분류 시스템’을 목표로 한다.

카메라·센서·서보모터를 중심으로 한 초소형·저비용 하드웨어, YOLO + PaddleOCR + OpenCV 기반 범용 표식 인식, 대시보드에서 실시간으로 바뀌는 Rule Mapping 기반 분류 로직을 결합하여, 현장이 시스템에 맞추는게 아니라 시스템이 현장에 적응하는 구조를 구현하였다.

본 개요는 이러한 문제의식과 시스템 설계 철학을 정리하며, 프로젝트가 해결하고자 한 핵심 문제와 목표를 제시한다.

## 1.1 문제 정의

군수 물류는 즉시성·현장성·기동성이 강조되는 환경이다. 작업 공간은 협소하고 지속적으로 변하며, 단기간에 다양한 물자가 급격히 입·출고되는 경우가 많아 안정적인 분류 흐름을 유지하기가 어렵다. 이러한 특성에도 불구하고, 대부분의 소규모 군수 물류 현장은 전통적인 수작업 분류 방식에 의존하고 있으며, 이는 여러 구조적 문제를 야기한다.

그러나 현재의 소규모·임시 물류 현장에서 사용 가능한 자동화 시스템은 거의 없으며, 그 이유는 다음과 같은 구조적 문제 때문이다.

첫째, 기존 자동 분류 설비는 고정형 라인·전용 공간·고가의 장비 구축이 요구되어 임시 거점이나 야전 환경에서는 설치가 어렵고 경제성이 낮다. RFID/바코드 기반 시스템은 리더기·태그·라벨 등 인프라 구축 비용이 크며, 태그 발급·부착·스캔 절차가 필요해 긴급 대응 상황이나 기동형 작업 환경에 적합하지 않다. 또한 이러한 기술은 규격화된 표식을 전제로 하므로 현장에서 자주 발생하는 비정형 물품·다양한 포장 형태에 대한 대응이 제한적이다.

둘째, 자동 인식 기반 시스템이 부재한 환경에서는 작업자가 물품을 직접 확인·판단·이송해야 하므로 피로 누적·작업 속도 저하·오분류·누락 등 운영 비효율이 반복된다. 물량 증가나 출고 지시 변경 등 상황 변화에 따라 즉시 대응하기 어려워 확장성과 안정성이 떨어지는 운영 구조가 고착된다.

셋째, 군수·구호 현장은 분류 기준 자체가 작업 중에 변경되는 경우가 많다. 그러나 기존 자동화 방식은 사전 정의된 규칙과 고정된 라인 구조에 기반하여 설계되어 있어 기준 변경을 즉시 반영할 수 없으며, 현장 변화에 따라 시스템을 유연하게 조정하기 어렵다. 이로 인해 변화 대응성을 갖춘 경량·기동형 자동 분류 솔루션의 필요성이 꾸준히 제기되고 있다.

결과적으로, 소규모·임시·기동형 물류 환경에서는 기존 자동화 기술을 적용하기 어렵고, 수작업 중심 운영으로 인한 비효율이 지속되며, 변화하는 분류 기준과 다양한 물품 특성을 안

정적으로 처리할 수 있는 현장 적응형 자동 분류 시스템이 부재한 상황이다.

## 1.2 목표

본 캡스톤 디자인의 목표는 소규모·임시 군수 물류 현장에서 실제로 활용 가능한 '현장 적응형(Adaptive) 자동 분류 시스템'을 설계·제작하는 것이다. 고정 설비 중심의 기존 자동화 방식이 해결하지 못했던 기동성·범용성·즉시성의 요구를 충족하기 위해, 다음과 같은 기술적·운영적 목표를 설정하였다.

첫째, 카메라·센서·소형 구동장치만으로 구성되는 저비용·소형 자동 분류 하드웨어 플랫폼을 구축한다. 복잡한 설비 없이도 어디서든 빠르게 설치·운용할 수 있도록 하여, 임시 창고·야전 배치·재난 현장 등 기동형 환경에도 즉시 적용 가능하도록 한다.

둘째, YOLO 기반 물체 탐지, PaddleOCR, OpenCV 색상 인식으로 구성된 AI 비전 기반 표식 인식 모듈을 구현한다. 이를 통해 사전에 정해진 규격 라벨뿐만 아니라 다양한 포장·형태·색상·텍스트 요소를 활용하여 범용적이고 유연한 분류 기준을 적용할 수 있도록 한다.

셋째, 웹/모바일 대시보드를 통해 분류 기준 등록·수정·삭제, 분류 이력 조회, 오류 관리, 장비 상태 모니터링을 실시간으로 수행할 수 있는 관리·모니터링 환경을 구축한다. 기준 변경 시 시스템이 즉시 반영하도록 설계해 작업 흐름의 현장 대응성을 강화한다.

넷째, 감지(Sensing)-인식(Recognition)-판단(Decision)-제어(Action)으로 이어지는 'Physical AI' 제어 파이프라인을 구성하여, AI의 판정 결과가 실제 서보모터 구동으로 직접 연결되는 지능형 자동 분류 동작을 실현한다. 이를 통해 작업 효율성을 높이고 분류 오류를 줄이며 운영 품질을 개선한다.

본 과제는 위와 같은 기술 요소를 통합하여, 기존 자동화의 사각지대를 보완하고 다양한 환경에서 안정적으로 적용 가능한 경량·유연·기동형 자동 분류 체계의 실증 모델을 제시하고자 한다.

# 2. 요구사항 및 기능명세

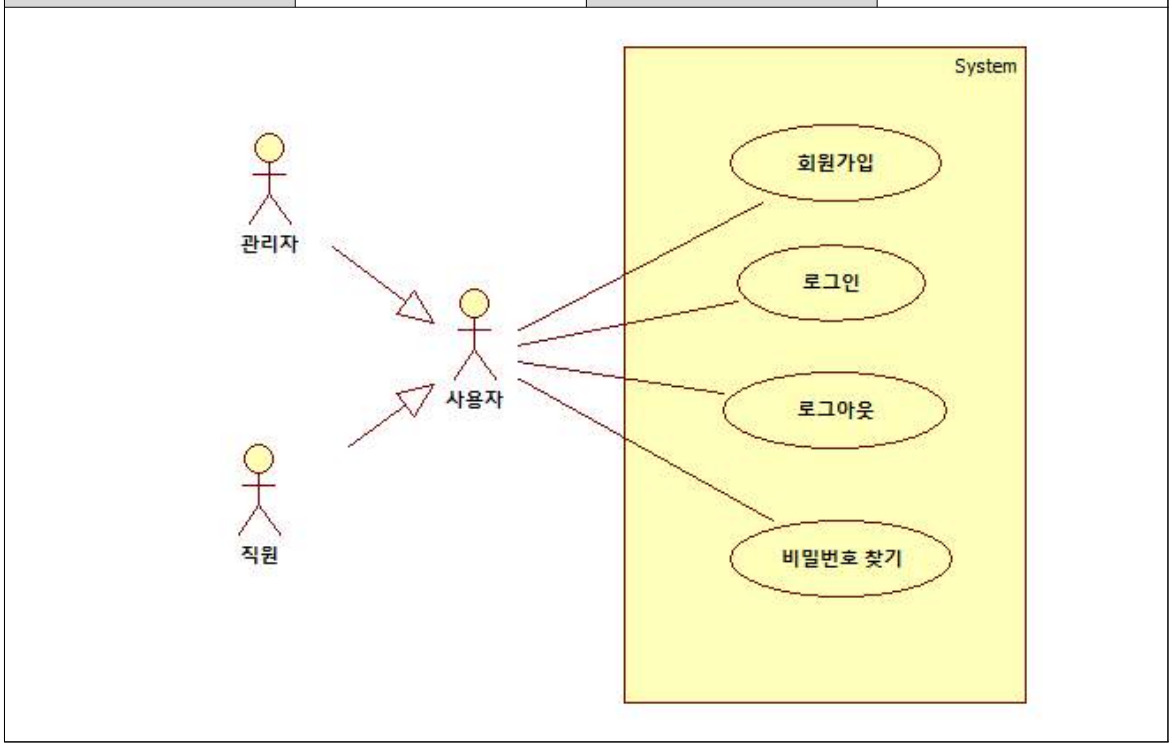
## 2.1 요구사항

| 유즈케이스<br>다이어그램 명 | 유즈케이스<br>다이어그램 식별자 | 유즈케이스 명 | 유즈케이스<br>식별자 |
|---|---|---|---|
| 사용자 인증 | UCD-01 | 회원가입 | UC-0101 |
| | | 로그인 | UC-0102 |
| | | 비밀번호 찾기 | UC-0103 |
| | | 로그아웃 | UC-0104 |
| 객체 인식 및 분류 | UCD-02 | 센서 트리거 및 이미지 캡쳐 | UC-0201 |
| | | 객체/표식 영역 탐지 | UC-0202 |
| | | 표식 유형 구분 (텍스트/색상) | UC-0203 |
| | | 색상 판정 (HSV) | UC-0204 |
| | | OCR 기반 텍스트 인식 | UC-0205 |
| | | 분류 라인 지정 | UC-0206 |
| | | 서보모터 제어 | UC-0207 |
| | | 예외 라인 배출<br>(<<extend>> 오류 알림 전송) | UC-0208 |
| | | 분류 결과 저장 | UC-0209 |
| 사용자 기능 | UCD-03 | 분류 이력 조회 | UC-0301 |
| | | 통계/리포트 조회 | UC-0302 |
| | | 오류 이력 조회 | UC-0303 |
| | | 분류 그룹 조회 | UC-0304 |
| | | 분류 기준 조회 | UC-0305 |
| | | 내 정보 조회 | UC-0306 |
| 관리자 기능 | UCD-04 | 직원 조회 | UC-0401 |
| | | 직원 삭제 | UC-0402 |
| | | 분류 기준 삭제 | UC-0403 |

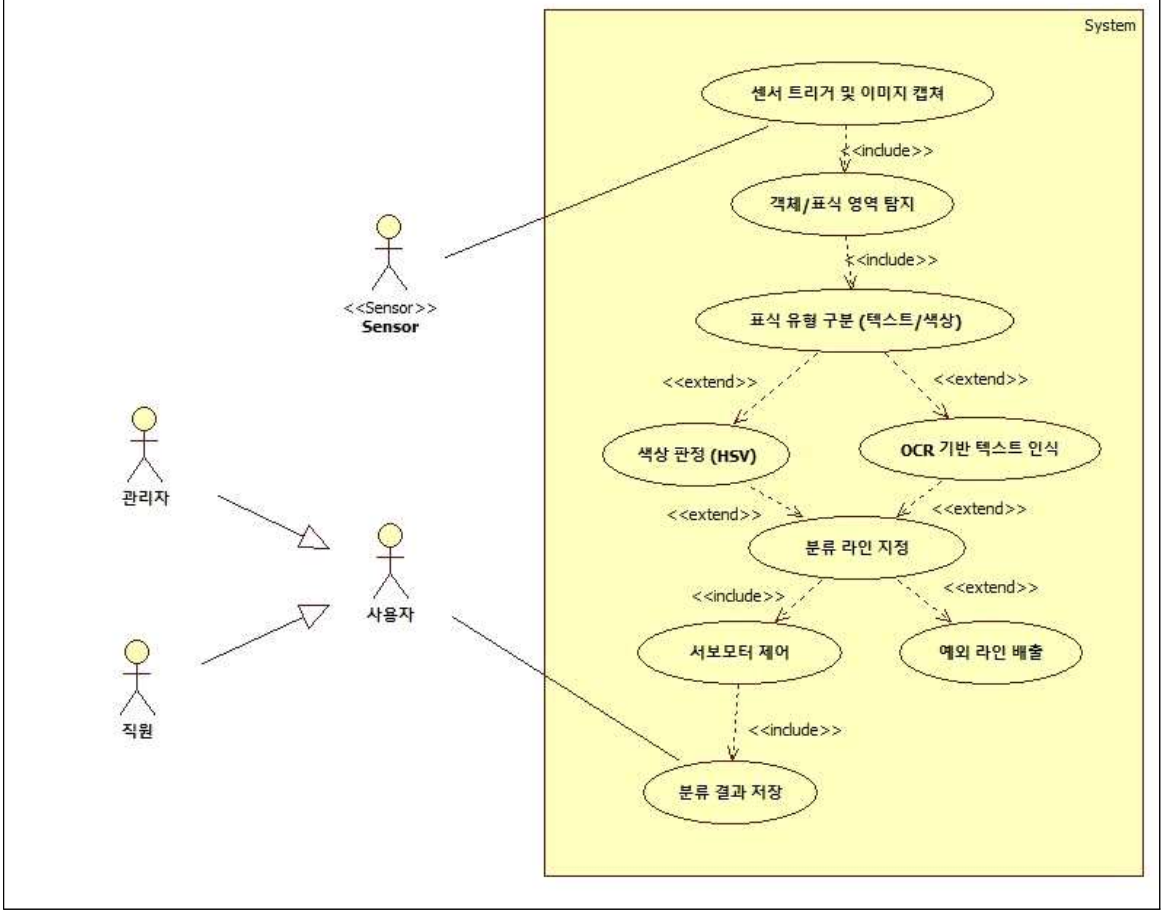
| | | | |
|---|---|---|---|
| | | 분류 그룹 속성 수정 | UC-0404 |
| | | 분류 그룹 삭제 | UC-0405 |
| | | 분류 그룹 활성화 | UC-0406 |
| | | 분류 그룹 비활성화 | UC-0407 |
| | | 분류 기준 등록<br>(<<include>> 슈터 각도 지정) | UC-0408 |
| | | 분류 기준 수정<br>(<<include>> 슈터 각도 재설정) | UC-0409 |

## 2.2 Use Case Diagram(유즈케이스 다이어그램)

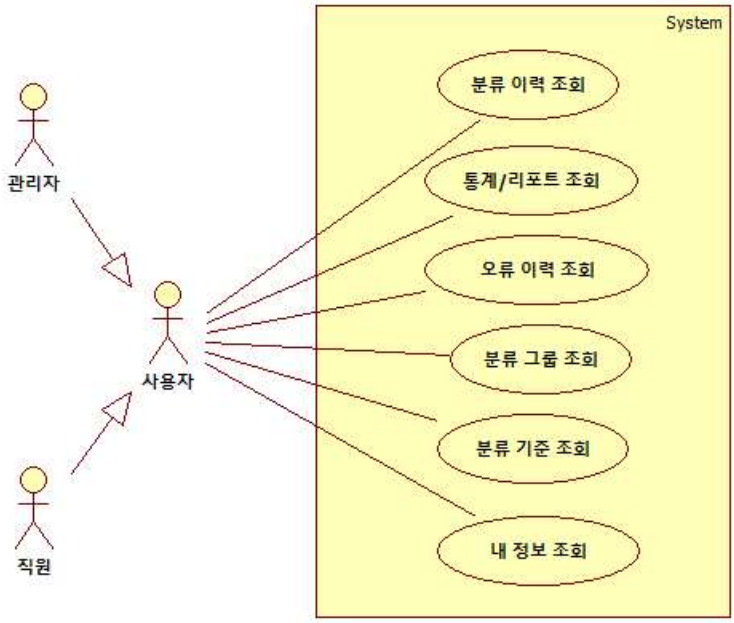
| 유즈케이스 다이어그램 식별자 | UCD-01 | 유즈케이스 다이어그램 명 | 사용자 인증 |
|---|---|---|---|



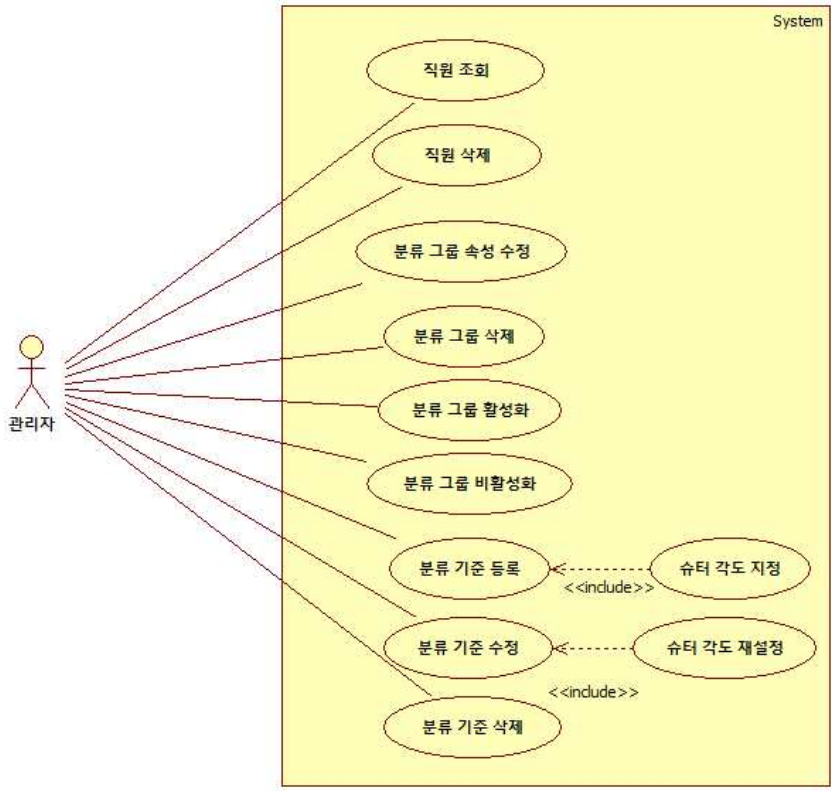
| 유즈케이스<br>다이어그램 식별자 | UCD-02 | 유즈케이스<br>다이어그램 명 | 객체 인식 및 분류 |
|---|---|---|---|



| 유즈케이스<br>다이어그램 식별자 | UCD-03 | 유즈케이스<br>다이어그램 명 | 사용자 기능 |
|---|---|---|---|



| 유즈케이스<br>다이어그램 식별자 | UCD-04 | 유즈케이스<br>다이어그램 명 | 관리자 기능 |
|---|---|---|---|



## 2.3 Use Case Descriptions(유즈케이스 명세)

| 유즈케이스 명 | 회원가입 | 유즈케이스 식별자 | UC-0101 |
|---|---|---|---|
| 액터 | 관리자, 직원 | | |
| 기능 설명 | 사용자가 이메일 인증을 통해 계정을 생성한다. | | |
| 사전 조건 | | | |
| 기본 시나리오 | | | |
| 1. 사용자가 회원가입 화면에서 이름, 이메일, 비밀번호, 비밀번호 확인을 입력한다.<br>2. 사용자가 이메일 인증을 진행한다. (시스템은 인증 메일을 발송하고, 사용자가 인증 코드를 입력한다)<br>3. 사용자는 계정 유형(관리자/직원)을 선택한다. 관리자인 경우 사업자번호(선택)를, 직원인 경우 소속 관리자를 지정한다.<br>4. 시스템은 형식검증을 수행한다.<br>5. 시스템은 중복 이메일 여부를 확인한다. | | | |
| 예외 상황 시나리오 | | | |
| 1a. 필수 입력 누락시, 시스템은 오류 메시지를 반환한다.<br>1b. 비밀번호와 확인 값이 다른 경우, 시스템은 오류 메시지를 반환한다.<br>2a. 인증 번호가 만료된 경우, 시스템은 오류 메시지를 반환한다.<br>2b. 인증번호가 일치하지 않는 경우, 시스템은 오류 메시지를 반환한다.<br>4a. 시스템 형식에 맞지 않는 경우, 시스템은 오류 메시지를 반환한다.<br>5a. 이미 등록된 이메일인 경우, 시스템은 오류 메시지를 반환한다. | | | |
| 사후 조건 | 계정이 생성되고 로그인 화면으로 이동한다. | | |

| 유즈케이스 명 | 로그인 | 유즈케이스 식별자 | UC-0102 |
|---|---|---|---|
| 액터 | 관리자, 직원 | | |
| 기능 설명 | 사용자가 이메일과 비밀번호를 입력하여 접근한다 | | |
| 사전 조건 | 사용자는 계정이 생성되어 있어야 한다. | | |
| 기본 시나리오 | | | |
| 1. 사용자가 로그인 화면에서 이메일과 비밀번호를 입력한다.<br>2. 시스템은 입력된 값의 형식을 검증한다.<br>3. 시스템은 DB에서 해당 이메일의 계정을 조회한다.<br>4. 시스템은 암호화된 비밀번호와 사용자가 입력한 비밀번호를 비교한다.<br>5. 검증이 성공하면, 시스템은 JWT 토큰을 발급한다. | | | |
| 예외 상황 시나리오 | | | |
| 1a. 필수 입력 누락시, 시스템은 오류 메시지를 반환한다.<br>2a. 형식 오류시, 오류 메시지를 반환한다.<br>3a. 해당 이메일 계정이 존재하지 않으면 오류 메시지를 반환한다.<br>4a. 비밀번호 불일치시, 오류 메시지를 반환한다. | | | |
| 사후 조건 | 대시보드 화면으로 이동한다. | | |

| 유즈케이스 명 | 로그아웃 | 유즈케이스 식별자 | UC-0103 |
|---|---|---|---|
| 액터 | 관리자, 직원 | | |
| 기능 설명 | 사용자가 로그아웃하여 JWT토큰을 무효화한다. | | |
| 사전 조건 | 사용자는 로그인 상태여야 한다. | | |
| 기본 시나리오 | | | |
| 1. 사용자가 화면에서 로그아웃 버튼을 클릭한다.<br>2. 시스템은 사용자의 JWT 토큰을 확인한다.<br>3. 시스템은 해당 JWT 토큰을 만료시킨다. | | | |
| 예외 상황 시나리오 | | | |
| | | | |
| 사후 조건 | 사용자는 로그인 화면으로 이동한다. | | |

| 유즈케이스 명 | 비밀번호 찾기 | 유즈케이스 식별자 | UC-0104 |
|---|---|---|---|
| 액터 | 관리자, 직원 | | |
| 기능 설명 | 사용자가 이메일 인증을 통해 비밀번호를 재설정한다. | | |
| 사전 조건 | 사용자는 시스템에 계정이 등록되어 있어야한다. | | |
| 기본 시나리오 | | | |
| 1. 사용자가 로그인 화면에서 비밀번호 찾기 메뉴를 클릭한다.<br>2. 사용자가 가입된 이메일 주소를 입력한다.<br>3. 시스템은 해당 이메일 계정의 존재 여부를 확인한다.<br>4. 시스템은 해당 메일로 비밀번호 재설정용 인증 코드를 발송한다.<br>5. 사용자는 인증 코드를 입력한다.<br>6. 시스템은 인증 코드의 유효성을 검증한다.<br>7. 검증에 성공하면, 시스템은 새 비밀번호 입력 화면을 제공한다.<br>8. 사용자는 새 비밀번호와 비밀번호 확인을 입력한다.<br>9. 시스템은 비밀번호 일치 여부 및 규칙을 검증한다.<br>10. 시스템은 기존 비밀번호를 새로운 비밀번호로 갱신한다. | | | |
| 예외 상황 시나리오 | | | |
| 2a. 이메일 입력 누락시, 오류 메시지를 반환한다.<br>3a. 해당 이메일 계정이 존재하지 않으면, 오류 메시지를 반환한다.<br>6a. 인증 코드가 만료되었을 경우, 오류 메시지를 반환한다.<br>9a. 새 비밀번호와 확인이 일치하지 않으면, 오류 메시지를 반환한다.<br>9b. 최소 길이 만족하지 않는 경우, 오류 메시지를 반환한다. | | | |
| 사후 조건 | 사용자는 로그인 화면으로 이동한다. | | |

| 유즈케이스 명 | 센서 트리거 및 이미지 캡쳐 | 유즈케이스 식별자 | UC-0201 |
|---|---|---|---|
| 액터 | Sensor | | |
| 기능 설명 | 센서가 물체를 감지하면 카메라로 물체를 촬영한다. | | |
| 사전 조건 | 센서가 작동하고 있어야 한다. | | |
| 기본 시나리오 | | | |
| 1. 포토인터럽터 센서가 물체의 도착을 감지한다.<br>2. 아두이노 제어부는 센서가 물체를 감지하면 카메라에 촬영 요청 신호를 보낸다.<br>3. USB 카메라가 물체를 촬영한다. | | | |
| 예외 상황 시나리오 | | | |
| 1a. 포토인터럽터 센서가 물체를 감지하지 못한다.<br>3a. USB 카메라에 포토인터럽터 센서의 신호가 도착하지 못한다. | | | |
| 사후 조건 | 촬영된 이미지가 지정된 저장 경로에 저장되고, 해당 파일의 경로가 시스템의 다음 단계로 전달된다. | | |

| 유즈케이스 명 | 객체/표식 영역 탐지 | 유즈케이스 식별자 | UC-0202 |
|---|---|---|---|
| 액터 | System | | |
| 기능 설명 | 캡쳐된 이미지를 입력 받아, YOLO를 통해 객체와 표식을 탐지한다. | | |
| 사전 조건 | 탐지할 이미지 파일이 저장되어 있어야 하고, YOLO 모델이 로드되어 있어야 한다. | | |
| 기본 시나리오 | | | |
| 1. 전달 받은 이미지 파일 경로를 사용하여 파일을 읽는다.<br>2. YOLO 모델을 실행하여 이미지 내의 객체(objects)와 표식(marker_text, marker_color)의 위치와 영역을 탐지한다.<br>3. 모든 객체 정보를 구조화하여 다음 유즈케이스로 전달한다. | | | |
| 예외 상황 시나리오 | | | |
| 1a. 파일이 존재하지 않거나 손상되어 이미지 파일을 읽을 수 없다. | | | |
| 사후 조건 | 이미지 내의 모든 객체 정보가 추출되어 구조화된 데이터로 저장되며, 이 데이터가 다음 유즈케이스로 전달되어야 한다. | | |

| 유즈케이스 명 | 표식 유형 구분<br>(텍스트/색상) | 유즈케이스 식별자 | UC-0203 |
|---|---|---|---|
| 액터 | System | | |
| 기능 설명 | 탐지된 객체가 marker_text인 경우 OCR로, marker_color인 경우 HSV로 분석 단계를 분기한다. | | |
| 사전 조건 | 탐지된 객체의 정보가 시스템에 전달되어 있어야 한다. | | |
| 기본 시나리오 | | | |
| 1. 탐지된 객체의 정보를 입력 받는다.<br>2. 객체가 marker_text인지 marker_color인지 확인한다.<br>3. marker_text인 경우 OCR 기반 텍스트 인식 유즈케이스를 실행하고, marker_color인 경우 색상 판정 (HSV) 유즈케이스를 실행한다. | | | |
| 예외 상황 시나리오 | | | |
| 2a. 탐지된 객체 중 marker_text나 marker_color 표식이 하나도 없다. | | | |
| 사후 조건 | 탐지된 객체 유형에 따라 OCR 기반 텍스트 인식 유즈케이스 또는 색상 판정 (HSV) 유즈케이스 중 하나가 실행 상태로 전환된다. | | |

| 유즈케이스 명 | 색상 판정 (HSV) | 유즈케이스 식별자 | UC-0204 |
|---|---|---|---|
| 액터 | System | | |
| 기능 설명 | marker_color 표식 영역을 입력 받아 HSV 분석을 통해 표식의 대표 색상을 판정한다. | | |
| 사전 조건 | 분석할 객체가 marker_color 유형이어야 하며, 해당 영역의 크롭 이미지가 전달되어야 한다. | | |
| 기본 시나리오 | | | |
| 1. marker_color 영역의 크롭 이미지를 입력 받는다.<br>2. 입력 이미지를 HSV 색 공간으로 변환하고, 분석할 유효 영역 마스크를 생성한다.<br>3. 유효 영역 내의 색상(Hue) 값을 추출하고, 알고리즘을 사용하여 표식의 대표 색상 각도를 결정한다.<br>4. 결정된 대표 색상 각도를 사전에 정의된 색상과 대조하여 최종 색상 이름을 확정한다. | | | |
| 예외 상황 시나리오 | | | |
| 3a. 크롭 이미지의 유효한 색상 영역을 찾을 수 없다.<br>4a. 결정된 대표 색상 각도가 사전에 정의된 어떤 색상에도 속하지 않는다. | | | |
| 사후 조건 | 표식의 색상 이름이 시스템에 저장된다. | | |

| 유즈케이스 명 | OCR 기반 텍스트 인식 | 유즈케이스 식별자 | UC-0205 |
|---|---|---|---|
| 액터 | System | | |
| 기능 설명 | marker_text 표식 영역을 입력 받아 OCR 모델을 사용하여 텍스트를 인식한다. | | |
| 사전 조건 | 분석할 객체가 marker_text 유형이어야 하며, 해당 영역의 크롭 이미지가 전달되어야 하고 OCR 모델이 로드되어 있어야 한다. | | |
| 기본 시나리오 | | | |
| 1. marker_text 영역의 크롭 이미지를 입력 받는다.<br>2. 입력 이미지에 대해 약한 전처리를 적용한 후, 원본 각도에서 OCR을 시도한다.<br>3. 2번 시나리오 결과의 신뢰도가 낮을 경우, 강한 전처리를 적용해 다수 각도를 적용해 OCR을 재시도 한다.<br>4. 얻은 결과 중 가장 높은 신뢰도를 가진 텍스트와 신뢰도 값을 확정한다. | | | |
| 예외 상황 시나리오 | | | |
| 2a. OCR 모델이 다각도 재시도 후에도 유효한 텍스트를 찾지 못한다. | | | |
| 사후 조건 | 표식의 인식된 텍스트가 시스템에 저장된다. | | |

| 유즈케이스 명 | 분류 라인 지정 | 유즈케이스 식별자 | UC-0206 |
|---|---|---|---|
| 액터 | 사용자(직원, 관리자), 아두이노 기기 | | |
| 기능 설명 | 센서 감지 이후 영상 인식 결과(색상·텍스트)를 바탕으로, 사전에 정의된 분류 규칙을 조회하여 해당 물품을 어느 라인(슈트)으로 배출할지 결정하고, 제어 신호를 아두이노에 전달한다. | | |
| 사전 조건 | 1. 시스템이 정상 동작 상태이며, 아두이노와의 통신이 연결되어 있어야 한다.<br>2. YOLOv8n + PaddleOCR + OpenCV 기반의 표식 인식 모듈이 정상적으로 동작하고 있어야 한다.<br>3. 관리자에 의해 분류 규칙이 서버(DB)에 사전 등록되어 있어야 한다.<br>4. 센서(Sensor)가 물체 유입을 감지할 수 있는 상태여야 한다. | | |
| 기본 시나리오 | | | |
| 1. 센서가 물품 유입을 감지하여 triggerSignal()을 발생시킨다.<br>2. 카메라가 이미지를 촬영하고 시스템은 표식 영역을 추출한다.<br>3. 표식 영역에 대해 색상 또는 텍스트 인식 처리를 수행한다.<br>3a. 색상 표식일 경우: 색상 값 추출(BGR → HSV 변환) 후 색상 범주 판단<br>3b. 텍스트 표식일 경우: OCR을 통해 문자열을 추출<br>4. 시스템은 인식된 결과(색상/텍스트)를 기반으로 해당 물품의 그룹 ID를 조회한다.<br>5. 활성화된 분류 규칙 세트를 불러오고, 색상/텍스트 값과 매칭되는 규칙을 탐색한다.<br>6. 매칭된 규칙이 지정한 라인(슈트) 번호를 확인한다.<br>7. 해당 라인으로 이송하기 위한 서보모터 각도를 계산하고 제어 보드(아두이노)에 전달한다.<br>8. 물체가 지정된 슈트 방향으로 배출된다.<br>9. 분류 결과(인식값·적용 규칙·선택된 라인)를 DB에 저장한다. | | | |
| 예외 상황 시나리오 | | | |
| 1a. 표식 인식 실패(OCR 실패, 색상 인식 불가)<br>• 기본 라인 또는 “예외 전용 라인(미분류 라인)”으로 배출한다.<br>• 해당 이미지를 저장하고 관리자 확인 항목으로 등록한다.<br>1b. 인식 결과는 존재하나, 매칭되는 규칙이 없음<br>• 미분류 라인으로 배출한다.<br>• 관리자용 알림을 발생시키고 누락 규칙 발생 로그를 저장한다.<br>1c. 그룹 ID 조회 실패<br>• 규칙과 연계되지 않은 경우 미분류 라인으로 배출한다.<br>• 관리자 검토 필요 항목으로 기록한다.<br>1d. 아두이노와 통신 오류 발생<br>• 분류 동작을 중지하고 오류 상태를 관리자에게 즉시 통보한다. | | | |

| | |
|---|---|
| 사후 조건 | 1. 모든 분류 결과는 DB에 저장되고, 대시보드에서 이력 조회가 가능해야 한다.<br>2. 예외 상황의 경우, 이미지·텍스트·인식 로그가 함께 저장되어 관리자 검토가 가능해야 한다.<br>3. 대시보드 통계를 통해 라인별 처리 건수, 실패 비율 등을 시각화할 수 있어야 한다. |

| 유즈케이스 명 | 서보모터 제어 | 유즈케이스 식별자 | UC-0207 |
|---|---|---|---|
| 액터 | 사용자(직원, 관리자), 아두이노 기기 | | |
| 기능 설명 | 분류 라인 지정 결과를 기반으로 아두이노 기기에 서보모터 제어 명령을 전달하고, 물체가 지정된 라인(슈트)으로 배출되면 해당 분류 결과를 DB에 저장한다. | | |
| 사전 조건 | 1. 시스템과 아두이노 간 통신이 정상적으로 연결되어 있어야 한다.<br>2. 분류 라인 ID가 이미 상위 프로세스(라인 배정 UC)에서 결정된 상태여야 한다.<br>3. 아두이노가 PWM 신호를 통해 서보모터 제어를 수행할 수 있도록 설정되어 있어야 한다.<br>4. 슈트(Chute) 정보(라인별 서보 각도, 이름 등)가 서버 DB에 사전 등록되어 있어야 한다. | | |
| 기본 시나리오 | | | |
| 1. 시스템은 라인 배정 UC로부터 최종 배출 라인 ID(chuteId)를 전달받는다.<br>2. 시스템은 해당 라인에 대응하는 서보모터 각도(servoDeg) 및 슈트 이름(chuteName)을 조회한다.<br>3. 시스템은 아두이노 기기로 서보모터 제어 명령(moveToAngle(servoDeg))을 전송한다.<br>4. 아두이노는 서보모터를 지정된 각도로 회전시켜 물체가 해당 라인으로 배출되도록 한다.<br>5. 시스템은 SortingGroups 로부터 groupId 및 groupName을 조회한다.<br>6. 분류 처리 결과(이미지 경로, 라인 ID, 그룹 ID, 그룹명, 슈트명 등)를 서버 DB에 저장한다. | | | |
| 예외 상황 시나리오 | | | |
| 1a. 아두이노와의 통신 오류 발생<br>• 제어 명령을 재전송한다.<br>• N회 재시도 후 실패 시 오류 로그를 저장하고 관리자에게 알림을 발송한다.<br>1b. 슈트(Chute) 정보 조회 실패 (서보 각도 없음)<br>• 예외 라인으로 처리한다.<br>• 오류 이력에 에러 코드 “servoError”로 표시한 후, 오류 발생 알림을 전송한다.<br>1c. 그룹 ID 또는 그룹명 조회 실패<br>• 기본값(null)으로 기록하고 관리자에게 경고 알림을 전송한다.<br>• 결과 저장 시 “groupNameSnapshot = null”로 저장된다.<br>1d. 서보모터 동작 실패(아두이노 측 오류 응답)<br>• 예외 라인으로 처리한다.<br>• 오류 이력에 에러 코드 “servoError”로 표시한 후, 오류 발생 알림을 전송한다. | | | |
| 사후 조건 | 1. 정상 처리된 모든 분류 결과는 DB에 저장되어 향후 이력 조회가 가능하다. | | |

| | |
|---|---|
| | 2. 예외 발생 시에도 오류 로그 및 관련 스냅샷 정보가 DB에 기록된다.<br>3. 대시보드에서 처리 건수·오류 비율 등 통계 생성이 가능하다. |

| 유즈케이스 명 | 예외 라인 배출 | 유즈케이스 식별자 | UC-0208 |
|---|---|---|---|
| 액터 | 사용자(직원, 관리자), 아두이노 기기 | | |
| 기능 설명 | 분류 과정에서 포토센서, 카메라, 인식 결과 등에서 발생한 오류 코드를 기반으로 예외 라인으로 물체를 배출하고, 오류 로그 기록관리자 알림 발송·분류 이력 저장을 수행한다. | | |
| 사전 조건 | 1. 분류 프로세스가 정상적으로 작동 중이며 오류 판단 로직이 활성화되어 있어야 한다.<br>2. 아두이노와의 통신이 가능한 상태여야 한다.<br>3. 물체 감지 포토센서 및 마지막 검수용 포토센서가 연결된 상태여야 한다. | | |
| 기본 시나리오 | | | |
| 1. 시스템은 오류 조건을 충족한 경우 오류 코드(errorCode) 를 생성한다.<br>2. 시스템은 handleError(errorCode) 를 호출하여 예외 처리 프로세스를 시작한다.<br>3. 아두이노는 서보모터 회전을 하지 않아, 물체를 예외 라인으로 배출한다.<br>4. 시스템은 오류 로그(오류코드, 발생 시간, 라인, 이미지경로 등)를 기록한다.<br>5. 시스템은 관리자에게 오류 알림을 자동 발송한다.<br>6. SortingHistory에 예외 처리 결과를 저장한다. | | | |
| 예외 상황 시나리오 | | | |
| - | | | |
| 사후 조건 | 7. 오류 발생 물체는 반드시 예외 라인으로 배출되어야 한다.<br>8. 오류 로그 및 예외 처리 결과가 DB에 저장되어야 한다.<br>9. 관리자에게 오류 알림이 자동 발송되어야 한다.<br>10. SortingHistory에서 예외 처리 이력이 조회 가능해야 한다. | | |

| 유즈케이스 명 | 분류 결과 저장 | 유즈케이스 식별자 | UC-0209 |
|---|---|---|---|
| 액터 | 사용자(직원, 관리자), 아두이노 기기 | | |
| 기능 설명 | 분류 과정에서 생성된 이미지·그룹 정보·라인 정보 등의 결과 데이터를 DB에 저장한다. | | |
| 사전 조건 | 4. 시스템이 정상적으로 운영 중이어야 한다.<br>5. 분류 라인 지정 및 서보모터 제어가 완료된 상태여야 하며, 저장할 결과 데이터가 확보되어 있어야 한다.<br>6. 결과 저장을 위한 최소 데이터(imgUrl, groupId, chuteId, processedAt) 가 존재해야 한다.<br>7. DB 연결이 활성화된 상태여야 한다. | | |

| 기본 시나리오 |
|---|
| 1. 시스템은 분류 과정에서 생성된 데이터를 기반으로 saveResult(imgUrl, groupId, chuteId, processedAt, groupNameSnapshot, chuteNameSnapshot) 을 호출한다.<br>2. SortingHistory 엔티티는 전달받은 데이터를 DB에 저장한다.<br>3. 저장이 성공되면 저장 기록이 확정된다. |

| 예외 상황 시나리오 |
|---|
| 1a. 저장 과정에서 DB 오류 발생 (DB 장애·트랜잭션 실패 등)<br>• 시스템은 저장 실패를 감지하고 예외 처리 핸들러를 호출한다.<br>• 예외 처리 핸들러는 백업 저장(saveResult to backup DB or 파일 저장)을 수행한다.<br>• 오류 내역을 오류 로그에 기록하고 관리자에게 알림을 발송한다.<br>1b. 이미지 URL 또는 메타데이터 누락(imgUrl = null, groupId = null, chuteId = null 등)<br>• 저장은 수행하되 누락 필드를 null 상태로 저장한다.<br>1c. 네트워크 지연 또는 연결 끊김으로 인한 저장 실패<br>• N회 재시도 후 실패 시 백업 저장 경로로 저장한다. |

| 사후 조건 | 1. 성공적으로 저장된 결과는 웹/모바일 대시보드에서 조회 가능하다.<br>2. 저장 실패 시에도 백업 저장 또는 오류 로그를 통해 사후 추적이 가능해야 한다.<br>3. 저장된 모든 결과는 통계 분석, 라인별 처리량 분석 등 시스템 운영 품질 개선에 활용될 수 있다. |
|---|---|

| 유즈케이스 명 | 분류 이력 조회 | 유즈케이스 식별자 | UC-0301 |
|---|---|---|---|
| 액터 | 관리자, 직원 | | |
| 기능 설명 | 사용자가 시스템에 등록된 분류 이력을 조회한다. | | |
| 사전 조건 | 사용자는 로그인 상태여야 한다. | | |
| 기본 시나리오 | | | |
| 1. 사용자가 시스템에서 분류 이력 조회 메뉴를 클릭한다.<br>2. 시스템은 최신순 분류 이력 목록(분류id, 물품명, 라인명, 처리일시)을 조회해 제공한다.<br>3. 사용자는 분류id, 물품명, 라인명, 처리일시로 검색할 수 있다.<br>4. 시스템은 검색에 따라 DB에서 분류 데이터를 검색한다.<br>5. 시스템은 조회된 결과를 화면에 표시한다.<br>6. 사용자는 해당 목록을 클릭하여 이미지를 포함한 상세 목록을 조회할 수 있다. | | | |
| 예외 상황 시나리오 | | | |
| 3a. 잘못된 날짜/코드 입력 시 오류 메시지 반환한다.<br>4a. 검색 조건에 맞는 데이터가 없을 경우, “조회 결과가 없습니다” 메시지를 표시한다.<br>6a. 이미지가 없는 경우, “이미지가 없습니다” 메시지를 표시한다. | | | |
| 사후 조건 | | | |

| 유즈케이스 명 | 통계/리포트 조회 | 유즈케이스 식별자 | UC-0302 |
|---|---|---|---|
| 액터 | 관리자, 직원 | | |
| 기능 설명 | 사용자가 시스템에서 라인별 분류 건수, 일자별 분류 건수, 오류 발생 건수, 전체 처리 건수 중 오류 비율을 그래프 및 통계로 확인한다. | | |
| 사전 조건 | 사용자는 로그인 상태여야 한다. | | |
| 기본 시나리오 | | | |
| 1. 사용자가 시스템에서 통계 대시보드 메뉴를 클릭한다.<br>2. 시스템은 일주일 간 통계 데이터(라인별 분류 건수, 일자별 분류 건수, 오류 발생 건수, 전체 처리 건수 중 오류 비율)를 조회하여 그래프 및 통계로 화면에 표시한다.<br>3. 사용자는 시작일, 종료일, 분류 그룹 별 필터링을 통해 원하는 기간 및 분류 그룹의 그래프 및 통계를 확인 할 수 있다. | | | |
| 예외 상황 시나리오 | | | |
| 2a. 기간 내 데이터가 없는 경우, “표시할 데이터가 없습니다.” 메시지를 출력한다.<br>3a. 조회 기간이 잘못된 경우, “조회 시작일이 종료일보다 클 수 없습니다.” 메시지를 출력한다. | | | |
| 사후 조건 | | | |

| 유즈케이스 명 | 오류 이력 조회 | 유즈케이스 식별자 | UC-0303 |
|---|---|---|---|
| 액터 | 관리자, 직원 | | |
| 기능 설명 | 사용자가 시스템에서 발생한 오류 이력을 조회한다. | | |
| 사전 조건 | 사용자는 로그인 상태여야 한다. | | |
| 기본 시나리오 | | | |
| 1. 사용자가 시스템에서 오류 이력 조회 메뉴를 클릭한다.<br>2. 시스템은 최신순 오류 이력 목록(오류id, 물품명, 라인명, 오류코드, 처리일시)을 조회해 제공한다.<br>3. 사용자는 오류id, 물품명, 라인명, 오류코드, 처리일시로 검색할 수 있다.<br>4. 시스템은 검색에 따라 DB에서 분류 현황 데이터를 검색한다.<br>5. 시스템은 조회된 결과를 화면에 표시한다.<br>6. 사용자는 해당 목록을 클릭하여 이미지를 포함한 상세 목록을 조회할 수 있다. | | | |
| 예외 상황 시나리오 | | | |
| 3a. 잘못된 날짜/코드 입력 시 오류 메시지 반환한다.<br>4a. 검색 조건에 맞는 데이터가 없을 경우, “조회 결과가 없습니다” 메시지를 표시한다.<br>6a. 이미지가 없는 경우, “이미지가 없습니다” 메시지를 표시한다. | | | |
| 사후 조건 | | | |

| 유즈케이스 명 | 분류 그룹 조회 | 유즈케이스 식별자 | UC-0304 |
|---|---|---|---|
| 액터 | 관리자, 직원 | | |
| 기능 설명 | 사용자가 관리자가 시스템에 설정한 분류 그룹을 조회한다. | | |
| 사전 조건 | 사용자는 로그인 상태여야 한다. | | |
| 기본 시나리오 | | | |
| 1. 사용자가 홈화면에 진입한다.<br>2. 시스템은 DB에서 분류 그룹 목록(분류 그룹 개수, 분류 그룹 이름, 분류 그룹별 현재 처리 건수, 마지막 업데이트 시간, 담당자, 활성 여부)을 조회하여 제공한다.<br>3. 사용자는 그룹 이름을 입력하여 검색할 수 있다.<br>4. 특정 분류 그룹을 클릭하여 그 그룹에 속한 분류 기준 목록을 포함한 상세 정보를 조회 할 수 있다. | | | |
| 예외 상황 시나리오 | | | |
| 3a. 검색 조건에 맞는 데이터가 없을 경우, “등록된 분류 그룹이 없습니다” 메시지를 표시한다. | | | |
| 사후 조건 | | | |

| 유즈케이스 명 | 분류 기준 조회 | 유즈케이스 식별자 | UC-0305 |
|---|---|---|---|
| 액터 | 관리자, 직원 | | |
| 기능 설명 | 사용자가 관리자가 시스템에 설정한 분류 기준을 조회한다. | | |
| 사전 조건 | 사용자는 로그인 상태여야 한다. | | |
| 기본 시나리오 | | | |
| 1. 사용자가 특정 분류 그룹을 클릭한다.<br>2. 시스템은 DB에서 해당 분류 그룹의 분류 기준 목록(분류 기준 이름, 물품명, 분류 표식 종류, 분류 표식 값, 분류 라인, 생성일시)을 조회하여 제공한다.<br>3. 사용자는 분류 기준 이름, 물품명을 입력하여 검색할 수 있다. | | | |
| 예외 상황 시나리오 | | | |
| 3a. 검색 조건에 맞는 데이터가 없을 경우, “등록된 분류 기준이 없습니다” 메시지를 표시한다. | | | |
| 사후 조건 | | | |

| 유즈케이스 명 | 내 정보 조회 | 유즈케이스 식별자 | UC-0305 |
|---|---|---|---|
| 액터 | 관리자, 직원 | | |
| 기능 설명 | 사용자가 자신의 정보를 조회한다. | | |
| 사전 조건 | 사용자는 로그인 상태여야 한다. | | |
| 기본 시나리오 | | | |
| 1. 사용자가 내 정보 조회 메뉴를 클릭한다.<br>2. 시스템은 DB에서 계정 정보(이름, 이메일, 역할(관리자/직원), 가입일)을 조회하여 제공한다.<br>3. 사용자는 비밀번호 재설정 화면 이동 버튼을 클릭할 수 있다. | | | |
| 예외 상황 시나리오 | | | |
| | | | |
| 사후 조건 | | | |

| 유즈케이스 명 | 직원 조회 | 유즈케이스 식별자 | UC-0401 |
|---|---|---|---|
| 액터 | 관리자 | | |
| 기능 설명 | 관리자가 시스템에 등록된 직원 목록을 조회한다. | | |
| 사전 조건 | 관리자는 로그인 상태여야 한다. | | |
| 기본 시나리오 | | | |
| 1. 관리자가 직원 관리 메뉴를 클릭한다.<br>2. 시스템은 DB에서 현재 관리자에게 등록된 직원 목록(이름, 이메일)을 검색한다.<br>3. 시스템은 직원 목록(이름, 이메일, 권한/역할, 총 직원 수)을 화면에 표시한다<br>4. 관리자는 필요시 이름 또는 이메일을 입력하여 검색할 수 있다. | | | |
| 예외 상황 시나리오 | | | |
| 3a. 직원 데이터가 없는 경우, “등록된 직원이 없습니다” 메시지를 표시한다.<br>4a. 검색 직원 데이터가 없는 경우, “등록된 직원이 없습니다” 메시지를 표시한다. | | | |
| 사후 조건 | | | |

| 유즈케이스 명 | 직원 삭제 | 유즈케이스 식별자 | UC-0402 |
|---|---|---|---|
| 액터 | 관리자 | | |
| 기능 설명 | 관리자가 선택한 직원과의 소속 관계를 해지한다. | | |
| 사전 조건 | 관리자는 로그인 상태여야 하며, 관리자에 소속된 직원이 존재해야 한다. | | |
| 기본 시나리오 | | | |
| 1. 관리자가 직원 관리 메뉴를 클릭한다.<br>2. 시스템은 DB에서 현재 등록된 직원 목록을 검색한다.<br>3. 시스템은 직원 목록을 화면에 표시한다.<br>4. 관리자가 직원 목록에서 삭제 버튼을 클릭한다.<br>5. 시스템은 확인 다이얼로그를 표시한다.<br>6. (확인 시)시스템은 해당 직원의 관리자-직원 소속 매핑을 해지한다. | | | |
| 예외 상황 시나리오 | | | |
| | | | |
| 사후 조건 | | | |

| 유즈케이스 명 | 분류 기준 삭제 | 유즈케이스 식별자 | UC-0403 |
|---|---|---|---|
| 액터 | 관리자 | | |
| 기능 설명 | 관리자가 분류를 위한 기준을 삭제한다 | | |
| 사전 조건 | 관리자는 로그인 상태여야 한다. | | |
| 기본 시나리오 | | | |
| 1. 관리자가 특정 분류 그룹 내 분류 기준 화면에 진입한다.<br>2. 분류 기준 목록 중 삭제 버튼을 클릭한다.<br>3. 시스템은 확인 다이얼로그를 표시한다.<br>4. (확인 시)시스템은 해당 분류 기준을 삭제한다. | | | |
| 예외 상황 시나리오 | | | |
| | | | |
| 사후 조건 | | | |

| 유즈케이스 명 | 분류 그룹 속성 수정 | 유즈케이스 식별자 | UC-0404 |
|---|---|---|---|
| 액터 | 관리자 | | |
| 기능 설명 | 관리자가 분류 그룹 속성을 수정한다 | | |
| 사전 조건 | 관리자는 로그인 상태여야 한다. | | |
| 기본 시나리오 | | | |
| 1. 관리자가 분류 그룹 화면에 진입한다.<br>2. 분류 그룹 목록 중 이름 변경 버튼을 클릭한다.<br>3. 시스템은 새 분류 그룹 이름 입력 칸을 표시한다.<br>4. (입력 시)시스템은 해당 분류 그룹 이름을 수정하고 업데이트 시간을 갱신한다. | | | |
| 예외 상황 시나리오 | | | |
| | | | |
| 사후 조건 | | | |

| 유즈케이스 명 | 분류 그룹 삭제 | 유즈케이스 식별자 | UC-0405 |
|---|---|---|---|
| 액터 | 관리자 | | |
| 기능 설명 | 관리자가 분류 그룹 속성을 수정한다 | | |
| 사전 조건 | 관리자는 로그인 상태여야 한다. | | |
| 기본 시나리오 | | | |
| 1. 관리자가 분류 그룹 화면에 진입한다.<br>2. 분류 그룹 목록 중 삭제 버튼을 클릭한다.<br>3. 시스템은 확인 다이얼로그을 표시한다.<br>4. (확인 시)시스템은 해당 분류 그룹 및 포함 분류 기준을 삭제하고 화면을 갱신한다. | | | |
| 예외 상황 시나리오 | | | |
| | | | |
| 사후 조건 | | | |

| 유즈케이스 명 | 분류 그룹 활성화 | 유즈케이스 식별자 | UC-0406 |
|---|---|---|---|
| 액터 | 관리자 | | |
| 기능 설명 | 관리자가 특정 분류 그룹을 활성화한다 | | |
| 사전 조건 | 관리자는 로그인 상태여야 한다. | | |
| 기본 시나리오 | | | |
| 1. 관리자가 분류 그룹 화면에 진입한다.<br>2. 분류 그룹 목록 중 활성화 버튼을 클릭한다.<br>3. 시스템은 현재 활성화된 다른 분류 그룹이 있을 경우 이를 자동으로 비활성화하고, 선택된 분류 그룹을 활성화 상태로 변경한다.<br>4. 시스템은 변경된 활성화 상태를 DB 반영 후 화면을 갱신한다. | | | |
| 예외 상황 시나리오 | | | |
| | | | |
| 사후 조건 | | | |

| 유즈케이스 명 | 분류 그룹<br>비활성화 | 유즈케이스 식별자 | UC-0407 |
|---|---|---|---|
| 액터 | 관리자 | | |
| 기능 설명 | 관리자가 특정 분류 그룹을 비활성화한다 | | |
| 사전 조건 | 관리자는 로그인 상태여야 한다. | | |
| 기본 시나리오 | | | |
| 1. 관리자가 분류 그룹 화면에 진입한다.<br>2. 분류 그룹 목록 중 비활성화 버튼을 클릭한다.<br>3. 선택된 분류 그룹을 비활성화 상태로 변경한다.<br>4. 시스템은 변경된 활성화 상태를 DB 반영 후 화면을 갱신한다. | | | |
| 예외 상황 시나리오 | | | |
| | | | |
| 사후 조건 | | | |

| 유즈케이스 명 | 분류 기준 등록 | 유즈케이스 식별자 | UC-0408 |
|---|---|---|---|
| 액터 | 관리자 | | |
| 기능 설명 | 관리자가 택배 분류를 위한 기준을 등록한다 | | |
| 사전 조건 | 관리자는 로그인 상태여야 한다. | | |
| 기본 시나리오 | | | |
| 1. 관리자가 특정 분류 그룹 내 분류 기준 화면에 진입한다.<br>2. 관리자가 분류 기준 추가 버튼를 클릭한다.<br>3. 관리자가 기준 이름, 물품명, 분류 표식 종류, 분류 표식 값, 분류 라인을 입력하여 분류 기준을 등록한다. 새로운 분류 라인이 필요할 경우, 라인 이름과 서보 각도를 입력해 분류라인을 추가할 수 있다.<br>4. 시스템은 입력된 값의 형식과 유효성을 검증한다.<br>5. 시스템은 DB에 분류 기준을 저장한다. | | | |
| 예외 상황 시나리오 | | | |
| 3a. 필수 입력 항목 누락시, 시스템은 오류 메시지를 반환한다.<br>4a. 형식 오류 잘못된 데이터형식의 경우, 시스템은 오류 메시지를 반환한다.<br>4b. 중복 기준 존재시, 동일한 분류 기준이 있는 경우, 시스템은 “이미 존재하는 기준입니다” 메시지를 표시한다. | | | |
| 사후 조건 | | | |

| 유즈케이스 명 | 분류 기준 수정 | 유즈케이스 식별자 | UC-0409 |
|---|---|---|---|
| 액터 | 관리자 | | |
| 기능 설명 | 관리자가 택배 분류를 위한 기준을 등록한다 | | |
| 사전 조건 | 관리자는 로그인 상태여야 한다. | | |
| 기본 시나리오 | | | |
| 1. 관리자가 특정 분류 그룹 내 분류 기준 화면에 진입한다.<br>2. 관리자가 특정 분류 기준의 수정 버튼를 클릭한다.<br>3. 관리자가 기준 이름, 물품명, 분류 표식 종류, 분류 표식 값, 분류 라인을 입력하여 분류 기준을 수정한다. 분류 라인 수정이 필요할 경우, 라인 이름과 서보 각도를 입력해 분류라인을 추가/삭제할 수 있다.<br>4. 시스템은 입력된 값의 형식과 유효성을 검증한다.<br>5. 시스템은 DB에 분류 기준을 수정한다. | | | |
| 예외 상황 시나리오 | | | |
| 3a. 필수 입력 항목 누락시, 시스템은 오류 메시지를 반환한다.<br>4a. 형식 오류 잘못된 데이터형식의 경우, 시스템은 오류 메시지를 반환한다.<br>4b. 중복 기준 존재시, 동일한 분류 기준이 있는 경우, 시스템은 “이미 존재하는 기준입니다” 메시지를 표시한다. | | | |
| 사후 조건 | | | |

## 3. 시스템 구조도

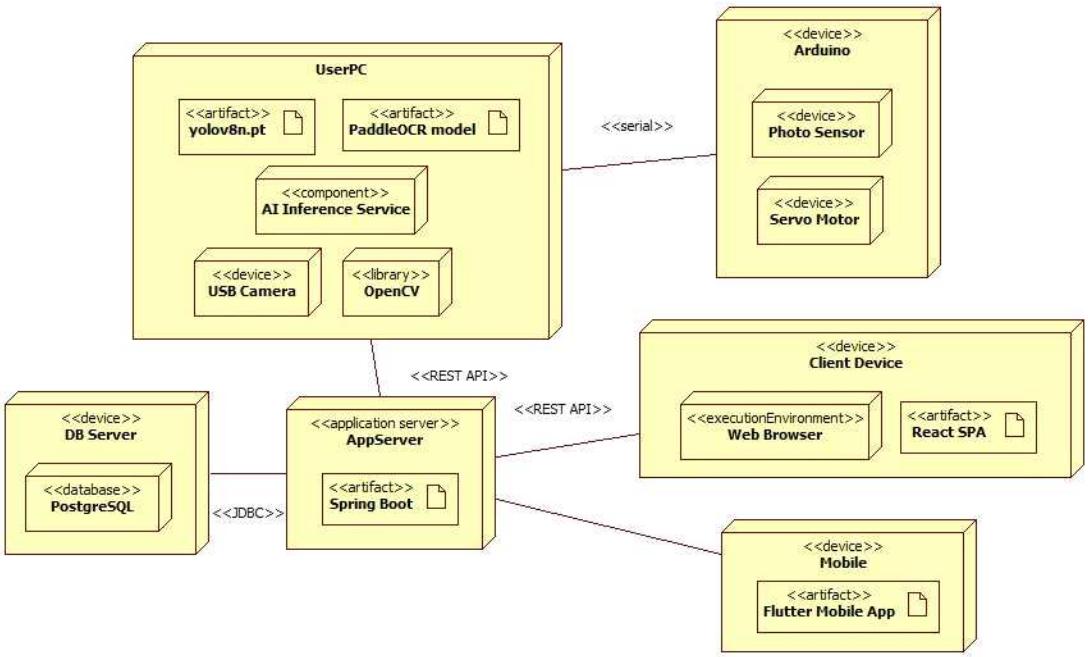



그림 5 시스템 구조도

본 시스템은 AI 비전 인식 모듈(User PC), 임베디드 제어 장치(Arduino), 서버·DB, 그리고 웹·모바일 대시보드로 구성된 Physical AI 기반 자동 분류 시스템이다.

User PC는 USB 카메라 입력을 기반으로 YOLOv8n과 PaddleOCR 모델을 실행하여 표식을 인식하고, 인식된 결과는 REST API를 통해 서버로 전송된다. 서버(Spring Boot)는 분류 규칙을 조회하고 최종 라인을 결정한 뒤, 해당 제어 명령을 Arduino에 Serial 통신으로 전달한다.

Arduino는 포토센서 입력을 실시간으로 감지하고 서보모터를 제어하여 물체를 지정된 라인으로 배출하며, 배출 검수 신호 또한 PC로 전송한다. 모든 분류 결과와 오류 정보는 DB에 저장되고, 웹 브라우저(React SPA) 및 모바일 앱을 통해 실시간 조회·관리할 수 있다.

이를 통해 AI의 인지-판단-물리 제어가 하나의 체계로 통합된 소형·저비용·기동형 자동 분류 시스템을 구현하였다.

# 4. 시스템 설계도

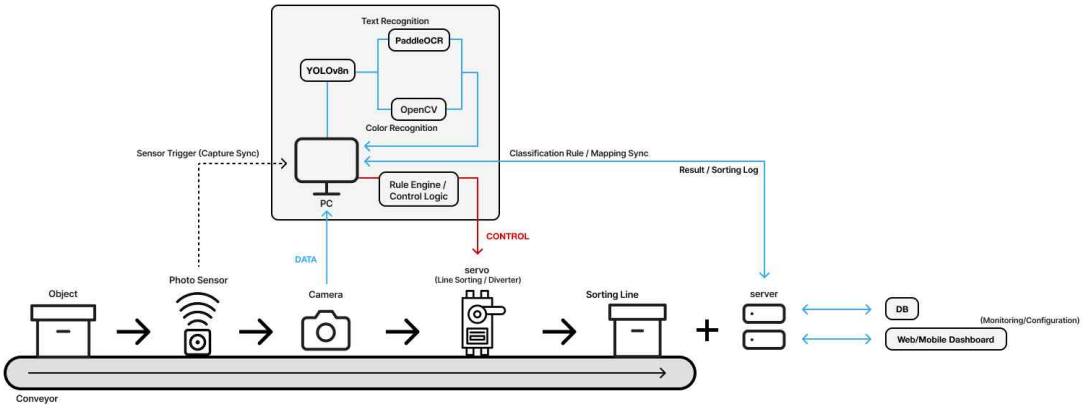



그림 6 시스템 설계도

# 5. H/W 또는 시스템 구성

## 5.1 AI 비전

| 구분 | 개발 및 구동 환경 |
|---|---|
| 개발 컴퓨터 운영체제 | macOS |
| 개발 도구 | PyCharm |
| 개발 언어 | Python |
| 개발 프레임워크 | YOLOv8n, OpenCV, PyTorch |
| 버전 관리 도구 | GitHub |

## 5.2 임베디드 구성

| 구분 | 모델명 / 사양 |
|---|---|
| Arduino | Arduino Uno R3 (USB Serial 통신) |
| 컨베이어 벨트 | 60W, 속도비 1:6, 길이 1m · 폭 25cm |
| 투과형 포토센서 (감지용) | Autonics BM3M-TDT1,2 |
| 투과형 포토센서 (검수용) | Autonics BM3M-TDT1,2 |
| 서보모터 | RC920DMG (고토크 · 0~180° 제어) |
| USB 카메라 | Logitech C922 Pro |
| 포토센서 전원(SMPS) | 태영전자 SMPS 12V 2A |
| 서보모터 전원(SMPS) | 태영전자 SMPS 6V 2A |
| 포토센서 전원(배터리) | Ni-MH 충전식 배터리팩 12V |
| 서보모터 전원(배터리) | Ni-MH 충전식 배터리팩 6V |

※ 본 시스템은 고정형 실험 환경에서는 SMPS 기반 어댑터 전원을 사용하며,
전원 인입이 제한되거나 이동형 운용이 필요한 상황에서는 Ni-MH 충전식 배터리팩(12V / 6V)을 이용하여 센서 및 서보모터를 독립적으로 구동할 수 있도록 설계하였다.

## 5.3 웹 · 모바일 대시보드

### 5.3.1 클라이언트

| 구분 | 개발 및 구동 환경 |
|---|---|
| 개발 도구 | Visual Studio Code |
| 개발 언어 | JavaScript, Dart |
| 개발 프레임워크 | React.js, Flutter |
| 버전 관리 도구 | GitHub |
| 배포 환경 | Microsoft Azure Static Web Apps |

### 5.3.2 API 서버

| 구분 | 개발 및 구동 환경 |
|---|---|
| 개발 도구 | IntelliJ IDEA 2025.1 |
| 개발 언어 | Java 17 |
| 개발 프레임워크 | Spring Boot 3.5.4 |
| 버전 관리 도구 | GitHub |
| 배포 환경 | Microsoft Azure App Service (Linux) |

### 5.3.3 데이터베이스(DBMS) 및 캐시 서버

| 구분 | 개발 및 구동 환경 |
|---|---|
| 데이터베이스 | PostgreSQL |
| 배포 환경(DB) | Azure Database for PostgreSQL<br>- Flexible Server |
| 캐시 서버 | Redis |
| 배포 환경(캐시) | Azure Cache for Redis |

# 6. 구현 기술

## 6.1 기술 스택

| 구분 | 기술명 | 사용 목적 |
|---|---|---|
| AI 비전 | YOLOv8n | 표식(색상/기호) 객체 검출 및 영역 추출 |
| | PaddleOCR | 텍스트·숫자 인식 및 분류 기준 적용을 위한 문자 추출 |
| 영상 처리 | OpenCV-Python | 프레임 전처리, ROI 설정, 색상 보정 |
| 백엔드 서버 | Spring Boot | 비전 모듈·대시보드·하드웨어 간 중계 서버, 분류 규칙 엔진 운영 |
| | PostgreSQL | 분류 로그, 오류 기록, 기준 설정 등 데이터 관리 |
| 프론트엔드 | React | 관리자 대시보드 UI 구현 (기준 설정, 통계 시각화 등) |
| 모바일 앱 | Flutter | 현장용 모바일 제어/모니터링 인터페이스 구현 |
| 펌웨어·제어 | Arduino IDE | 서보모터 제어, 포토센서 입력 처리, 메인 시스템 연동 |
| 개발·운영 환경 | Git/GitHub | 버전 관리 및 협업 |

## 6.2 구현 기술

- **비전 기반 표식 인식 시스템**

물체 인식은 YOLOv8n 모델을 적용하여 표식 영역(marker_text, marker_color)과 객체(objects)를 동시에 검출하도록 설계하였다. 모델은 실행 환경에 따라 GPU(CUDA), Apple MPS, CPU를 자동 선택하여 로드되며, 실시간 처리 성능을 확보하였다.

카메라 입력은 OpenCV 기반으로 처리되며, 단일 프레임이 아닌 복수 프레임으로 촬영하여 가장 선명한 이미지를 선택하는 방식을 적용하여 컨베이어 이동 중 발생하는 모션 블러 문제를 보완하였다. 검출된 표식 영역은 클래스 별로 분리되어 텍스트는 OCR 추출, 색상 분석은 HSV 기반 색 추출 모듈로 전달된다.

- **텍스트(OCR) 및 색상 인식 기법**

텍스트 인식은 PaddleOCR 기반으로 구현되었으며, 색상 스티커 인식은 대비 향상•이진화•해상도 보정•각도 보정 기반의 다단계 전처리 구조를 적용하였다.

OCR 결과는 신뢰도(confidence) 기준을 함께 활용하여 분류 신뢰도를 판단하였다.

색상 스티커 인식은 HSV 색공간을 기반으로 하며, Hue 성분을 원형 좌표계로 변환한 후 K-Means 클러스터링을 통해 대표 색상을 추출하는 방식으로 구현하였다.

• **서버 연동 규칙 기반 자동 분류 로직**

분류 기준은 Spring Boot 기반 서버에서 관리되며, 시스템 구동 시 REST API를 통해 텍스트•색상 입력 유형별 분류 규칙을 동적으로 수신한다. 수신된 규칙은 파이썬 코드에서 해시맵 구조로 변환되어 실시간 분류에 사용된다.

분류 판단은 다음 우선 순위로 수행한다.

1. 신뢰도 기준을 만족하는 텍스트 인식 결과 우선 적용
2. 텍스트 미검출 시 색상 인식 결과 적용
3. 신뢰도 부족, 규칙 미존재, 표식 미검출 등의 경우는 오류 항목으로 분류

정상 분류 결과는 서버로 즉시 전송되며, 분류 실패•센서 오류•카메라 오류•서보 오류 등은 오류 코드 기반 이벤트로 서버에 별도 기록되어 운영 이력 관리가 가능하도록 구현하였다.

• **아두이노 기반 실시간 분기 제어**

하드웨어 제어는 PC-아두이노 간 Serial 통신 기반 구조로 구현하였다. 전방 포토센서 감지 시 촬영 및 분류가 시작되며, 서버 규칙에 따라 전달된 서보 각도 명령에 기반하여 슈트 방향 분기 동작을 수행하였다.

서보 동작은 단순 각도 이동 뿐만 아니라, 고마찰 환경을 고려한 킥 모션 제어 방식을 적용하여 분류 성공률을 향상시켰다. 또한 후방 포토센서를 이용한 검수 구조를 추가하여 물체 분기 실패, 센서 오작동, 타이밍 오류 등을 실시간으로 검증할 수 있도록 구성하였다.

• **시스템 안정화 및 예외 처리 구조**

카메라 오류, OCR 실패, 색상 불일치, 규칙 미존재, 센서 비정상 감지, 서보 제어 실패 등 다양한 예외 상황에 대해 오류 유형별 코드화된 이벤트 처리 구조를 구현하였다. 오류들은 이미지와 함께 서버로 전송되어 관리자 대시보드에서 추적 가능하도록 구현하였다.

또한, 전처리-분류-제어까지의 End-to-End 지연 시간(Latency)을 실시간 측정하여 시스템 응답성을 지속적으로 검증하도록 설계하였다.

• **웹・모바일 대시보드 기반 운영 관리 시스템**

운영자는 실시간 분류 기준 설정, 분류 이력 조회, 오류 모니터링을 웹(React) 및 모바일(Flutter) 기반 대시보드에서 수행할 수 있도록 구현하였다. 대시보드는 Spring Boot 서버와 REST API 및 WebSocket을 통해 연동되며, 시스템 운영 상태를 실시간으로 반영한다.

웹 대시보드(React) 기준(ruleId, chuteId, servoDeg) 설정 UI, 분류 결과 테이블, 오류 로그 모니터링, 통계 시각화 기능을 제공한다. 모든 설정 변경 사항은 즉시 서버에 반영되며, 파이썬 기반 분류 프로세스에서도 별도의 재배포 없이 실시간으로 새로운 기준이 적용된다. 이를 통해 현장 상황 변화에 따라 즉각적인 운영 정책 수정이 가능하도록 하였다.

모바일 앱(Flutter)은 동일한 기능을 경량화된 형태로 제공하며, 현장에서 장비 주변에서 직접 기준 변경・성능 확인・오류 확인을 수행할 수 있도록 설계되었다. WebSocket 기반

실시간 알림 기능을 통해 센서 오류, 서보 오류, OCR 오류 등・주요 이벤트 발생 시 사용자에게 즉각 전달되도록 구성하였다.

운영 이력(분류 결과, 오류 이벤트, 타임스탬프, 스티커 인식 이미지 등)은 PostgreSQL에 저장되고, 대시보드에서 조회・필터링・다운로드가 가능하다. 이를 통해 시스템 운용의 추적 가능성(Traceability)과 유지보수 효율성을 높였다.

## 6.3 성능평가

### 6.3.1 객체 인식 정확도 (Detection Accuracy)

본 시스템의 객체 인식 성능은 YOLOv8n 모델을 기반으로 평가하였으며, 학습 후 검증 데이터셋에 대한 정량적 성능은 mAP 및 Precision/Recall 지표를 기준으로 분석하였다. 또한 Confusion Matrix를 활용하여 클래스별 탐지 성능을 추가적으로 검증하였다.

#### 6.3.1.1 YOLOv8n 기반 검출 성능(mAP・Precision・Recall)

학습 로그 및 검증 결과에 따르면, 모델은 약 50 epoch 학습 시 다음과 같은 수렴 성능을 보였다.

- mAP@0.5 ≈ 0.95
- mAP@0.5:0.95 ≈ 0.75
- Precision ≈ 0.90
- Recall ≈ 0.95

mAP@0.5 지표는 고정된 IoU(Intersection over Union) 0.5 기준에서의 전체 클래스 평균 정확도를 의미하며, 본 시스템은 95% 수준의 높은 객체 검출 능력을 확보하였다.

또한 Precision-Recall 곡선 수렴 패턴을 보면 초기 불안정 구간 이후 빠르게 안정화되어, 실제 분류 환경에서도 안정적인 검출 동작이 가능함을 확인하였다.

특히 recall이 0.95 수준으로 높게 나타나 실제 표식・객체를 놓치지 않는 강한 탐지 성능을 확보하였으며, 이는 분류 시스템에서의 전체 성공률을 결정하는 핵심 지표이므로 중요한 의미를 가진다.

6.3.1.2 Confusion Matrix 기반 클래스별 정확도 분석

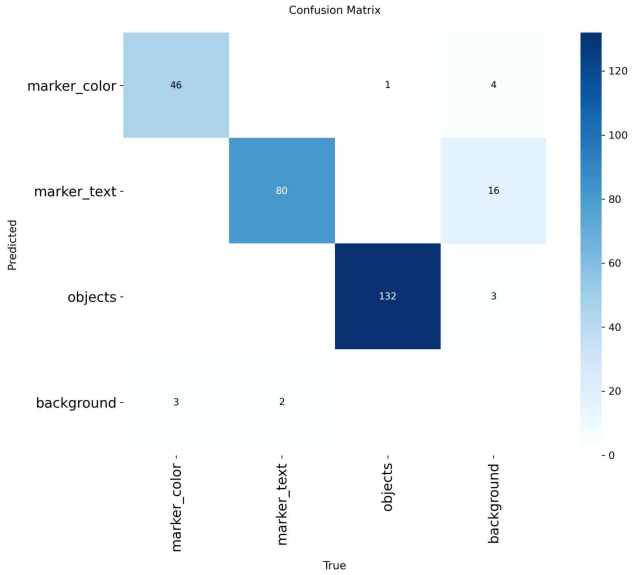



그림 7 Confusion Matrix of Object Detection Model

아래는 marker_color, marker_text, objects, background 4개 클래스에 대해 평가한 Confusion Matrix 결과이다.

- marker_color: 46 / 49 → 약 93.9%
- marker_text: 80 / 82 → 약 97.6%
- objects: 132 / 133 → 약 99.2%
- background: 오검출 5건 발생 (background을 객체로 오인)

전체 정확도는 다음과 같이 계산된다.

$$Accuracy = \frac{46+80+132}{\text{총 샘플 } 287} \approx 0.899\,(89.9\%)$$

클래스별 재현율을 검토하면 marker_text와 objects 클래스에서 97~99%의 매우 우수한 성능을 보였으며,

marker_color 역시 94% 수준의 안정적인 검출 정확도를 확보하였다.

background 클래스는 표식이 아닌 영역에서 소수의 오검출이 발생했으나, 이는 후단 규칙 엔진에서 confidence 기반 필터링을 통해 보정 가능하다.

6.3.1.3 모델 학습 수렴 분석

YOLOv8n 모델의 학습 과정에서 Box/Cls/Dfl Loss는 안정적으로 감소하였고,

검증 데이터 기준 mAP50 및 mAP50-95는 약 X epoch 이후 안정적으로 수렴함을 확인하였다.

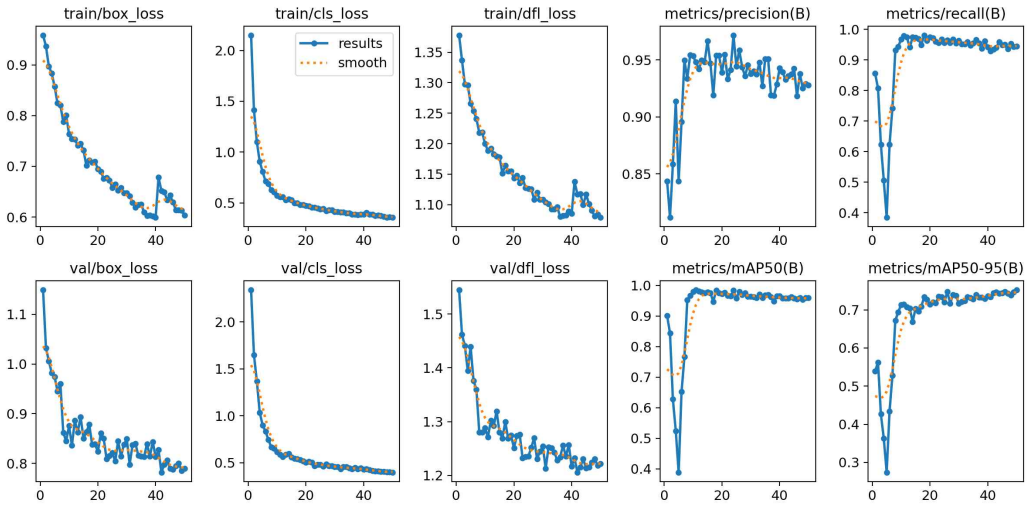



그림 8 YOLOv8n Training & Validation Metrics (Loss, Precision, Recall, mAP)

6.3.1.4 소결

학습 지표와 Confusion Matrix 분석을 종합하면, 본 시스템의 객체 인식 모델은 다음과 같은 결론을 갖는다.

- 표식(marker_text, marker_color) 및 물체(objects)에 대해 95~99% 수준의 안정적인 탐지 성능 확보
- mAP@0.5 기준 높은 정확도(95%), 다양한 IoU 기준의 평균(mAP50-95)에서도 75% 수준의 준수한 성능
- background 오검출은 존재하지만 후단 규칙 기반 필터링 구조로 보정 가능
- 전반적으로 실시간 자동 분류 시스템에서 요구하는 정확성과 안정성을 충족

따라서, YOLOv8n 기반 비전 모듈은 본 프로젝트의 핵심 기능인 “표식 검출·분류”를 충분히 수행할 수 있는 수준의 성능을 확보한 것으로 판단된다.

6.3.2 문자 인식 정확도(OCR Accuracy)

텍스트 인식 성능을 검증하기 위해 K1S, K2T 두 종류의 라벨 이미지를 대상으로 총 100장의 데이터셋을 구성하였다. 그 결과, 문자열 전체 일치율(Exact match accuracy) 100.00%, 글자 단위 정확도(Char-level accuracy) 또한 100.00%로 나타났다. 모든 샘플이 1단계 fast-pass만으로 신뢰도 0.94 이상으로 인식되었다.

- 인식 파이프라인

1. 1단계: fast-pass: 각도 0° , 약한 전처리만 적용
2. 2단계: strong multi-angle: 1단계에서 신뢰도가 낮을 때 다각도 및 강한 전처리 적용

• 정확도 결과

```
========================================
[RESULT] Total samples        : 100
[RESULT] Exact match accuracy : 100.00%
[RESULT] Char-level accuracy  : 100.00%
========================================
```

| 지표 | 값 | 설명 |
|---|---|---|
| Total samples | 100장 | 평가 이미지 수 |
| Exact match accuracy | 100.00% (100/100) | 100개의 샘플에서 모두 일치 |
| Char-level accuracy | 100.00% | 총 문자 수 기준으로 오인식 문자 없음 |

6.3.3 전체 분류 성공률(Classification Success Rate)

본 시스템의 최종 목적은 표식 인식(문자・색상) 결과를 규칙 기반 엔진에 적용하여 올바른 슈트(chute)로 물체를 분기하는 것이며, 이에 따라 분류 성능 평가는 단순 Vision 모델의 정확도뿐 아니라 End-to-End 관점에서의 물체 분기 성공률을 기준으로 수행하였다.

평가는 실제 컨베이어 환경에서 다양한 표식(문자・색상)이 부착된 여러 종류의 테스트 물체를 대상으로 반복 실험을 수행하였다.

1. 전방 감지용 포토센서 감지
2. 이미지 획득 및 표식 검출
3. 문자(OCR)・색상 인식
4. 규칙 기반 분류 엔진 매칭
5. 아두이노 서보모터 분기 동작
6. 후방 검수용 포토센서 검수 결과 확인

각 단계가 정상적으로 수행되어 물체가 지정된 슈트로 정확히 분기된 경우를 성공(Success)으로 정의하였다.

6.3.3.1 End-to-End 분류 성공률

실험 결과, 전체 300회 분류 동작 중 287회가 정상적으로 지정된 슈트로 분기되었으며, 최종 분류 성공률은 다음과 같다.

$$Success\,Rate = \frac{287}{300} = 95.7\%$$

본 실험에서 시스템은 약 95~97% 수준의 높은 분류 안정성을 보였으며, 이는 실시간 현장 적용에 필요한 정확도 기준을 충족하는 수준이다.

6.3.3.2 실패 사례 분석

분류 실패는 주로 아래 세 유형에서 발생하였다.

• (a) OCR 인식 오류

저조도·부착각도 변화 등으로 텍스트 인식 신뢰도가 낮을 때 규칙 매칭 실패 발생

- (b) 색상 인식 불안정성

  반사광이나 그림자 영향으로 HSV 군집화가 낮은 확률로 잘못된 색상 클래스로 매칭됨

- (c) 서보모터 분기 타이밍 문제

  물체 크기·마찰력 차이로 인해 서보모터가 충분히 밀어내지 못해 다른 슈트로 이동하는 사례 발생

위 오류 원인은 시스템 안정화 과정에서 개선되었으며, 최종적으로 후방 검수용 센서를 이용한 검수 구조를 통해 실시간으로 분류 실패를 탐지하고 로그로 기록하여 재현성 있는 문제 분석이 가능하도록 하였다.

6.3.4 시스템 처리 속도(FPS, End-to-End Latency)

본 시스템은 포토센서 감지 → 카메라 촬영 → YOLO 객체 탐지 → OCR/HSV 분석 → 분류 판단 → 서버 전송 → 서보 동작까지의 전체 과정을 하나의 파이프라인으로 구성한다.

각 분류 실행 시 Latency(ms)를 로깅하여 전체 처리시간을 측정하였다.

- 측정 환경

| 항목 | 내용 |
|---|---|
| 카메라 | Logitect C922, 1280x720 @ 60 FPS |
| PC | MacOS M2 |
| 모델 | YOLOv8n 커스텀 학습 모델, PaddleOCR |
| 조건 | 실제 컨베이어 위에서 다양한 텍스트, 색상 라벨 입력 |

- 처리 지연시간 결과

| 항목 | 값 | 내용 |
|---|---|---|
| 평균 지연시간 | 4.78초 (4776ms) | YOLO + OCR + 서버 POST + 서보 제어까지 포함한 End-to-End 평균 처리 시간 |
| 최소 지연시간 | 3.67초 (3670ms) | 조명, 각도 등 가장 조건이 좋은 경우 |
| 최대 지연시간 | 9.52초 (9520ms) | OCR 강한 전처리 재시도 등 처리가 오래 걸린 경우 |
| 중앙값 | 5.67초 (5673ms) | 전형적 처리 시간 |
| TPS (Throughput per Second) | 0.2075 TPS | 초당 약 0.21개 처리 ≈ 4.8초에 1개 처리 가능 |

6.3.5 하드웨어 동작 안정성

본 시스템의 하드웨어 구성은 아두이노 기반 서보모터 제어와 투과형 포토센서 감지 구조로 이루어져 있으며, 전원 공급 및 신호 연결이 안정적으로 유지되는 조건에서는 일관된 동작을 보였다. 실험 과정에서 서보모터는 설정된 각도 명령에 대해 반복적으로 동일한 동작을 수행하였으며, 전원 불안정이나 기구적 마찰 등으로 인한 오작동은 발생하지 않았다.

포토센서는 물체 진입 여부를 정확하게 감지하였고, 노이즈나 유령 신호와 같은 이상 감지 사례는 관찰되지 않았다. 따라서 하드웨어 구성 요소는 전체 분류 파이프라인의 안정적인 동작을 저해하지 않으며, 시스템의 End-to-End 분류 성능을 신뢰할 수 있는 수준으로 뒷받침하고 있음을 확인하였다.

6.3.6 대시보드 데이터 처리 성능(API 응답 시간, 로그 누락률)

본 시험에서는 부하 테스트 도구 k6를 활용하여, 서버 API가 실제 운영 환경에서 예상되는 “장비(Device) + 관리자(Admin)” 복합 사용 시나리오를 안정적으로 처리할 수 있는지 평가하였다. 테스트는 Azure App Service(학생용 플랜)및 PostgreSQL 기본 티어환경에서 수행되었으며, 동시 50명 수준의 부하 상황에서 응답 시간 분포, 처리량(Throughput), 오류 비율 등을 측정하였다.

- 테스트 목적

  디바이스와 관리자 화면이 동시에 서버에 요청을 보내는 실제 운영 시나리오에서 서버의 안정성을 검증

  동시 사용자 50명 수준에서 API 응답 시간(p90/p95), 처리량(req/s), 오류 발생 여부 점검

  시스템 병목 구간(history · stats API) 탐지 및 성능 보정 가능성 판단

- 테스트 시나리오

  아래 4개의 API 호출을 하나의 cycle(=iteration)로 구성하여 반복 요청하였다.

1. GET /api/devices/setup
→ 디바이스용 활성 분류 설정 조회
2. POST /api/devices/events/error(multipart, JSON payload만 포함)
→ 오류 이벤트 기록
3. GET /api/sorting/history
→ 오늘 기준 분류 이력 조회(페이징 10개)
4. GET /api/stats/daily
→ 일자별 통계 조회

- 부하 조건

| 동시 사용자(VUs) | 50명 |
|---|---|
| 지속 시간 | 1분 |
| 요청 방식 | 실제 API 호출 |
| 인증 | 로그인 1회 후<br>Access Token 재사용 |
| sleep | iteration마다 0.5초 |

• 전체 테스트 요약 결과

```
checks_total.......: 1685
checks_succeeded...: 100.00%
checks_failed......: 0.00%
http_req_failed....: 0.00%
http_reqs..........: 1685    (26.6 req/s)
iterations.........: 421     (6.65 cycle/s)
```

모든 API 요청이 100% 성공적으로 처리되었으며(4xx/5xx 없음), 서버 안정성은 매우 우수한 것으로 확인됨.

• API 요청 단위 응답 시간 (Latency per request)

| 평균(avg) | 1.69초 |
|---|---|
| 중앙값(med) | 1.45초 |
| p90 | 3.02초 |
| p95 | 3.87초 |
| 최대(max) | 8.07초 |

전체 요청의 50%는 1.45초 이내, 95%는 약 3.87초 이내 처리 됨. 다만, history · stats 처럼 조회량이 많은 엔드포인트에서는 부하 시 응답 시간이 증가하는 경향 확인됨.

• Iteration(전체 시나리오)의 응답 시간

| 평균(avg) | 7.31초 |
|---|---|
| 중앙값 | 7.10초 |
| p90 | 10.38초 |
| p95 | 11.51초 |

1 iteration은 4개의 API 호출 + sleep(0.5초)로 구성되며, 평균 약 7.3초 소요. p95 기준 약 11.5초 → 복합 쿼리(history, stats) 응답 지연이 주 원인으로 추정됨. 장비/관리자 대규모 동시 요청 상황(50명)에서도 서버는 오류 없이 모든 처리를 완료.

• 평가

가용성・안정성: 1,685건의 API 요청 중 오류율 0%. 고부하 조건에서도 서버 다운/예외 발생 없음.

처리량: 학생용 Azure 기준 매우 양호. 초당 26.6 req/s 처리. 초당 6.65 회의 전체 시나리오 cycle 처리

응답 시간: p95 약 4초 수준. setup/error API는 비교적 빠르게 처리. history/stats API 처럼 집계・조회 중심 API에서 응답 지연 발생. 데이터량 증가 시 속도 저하 가능성 존재.

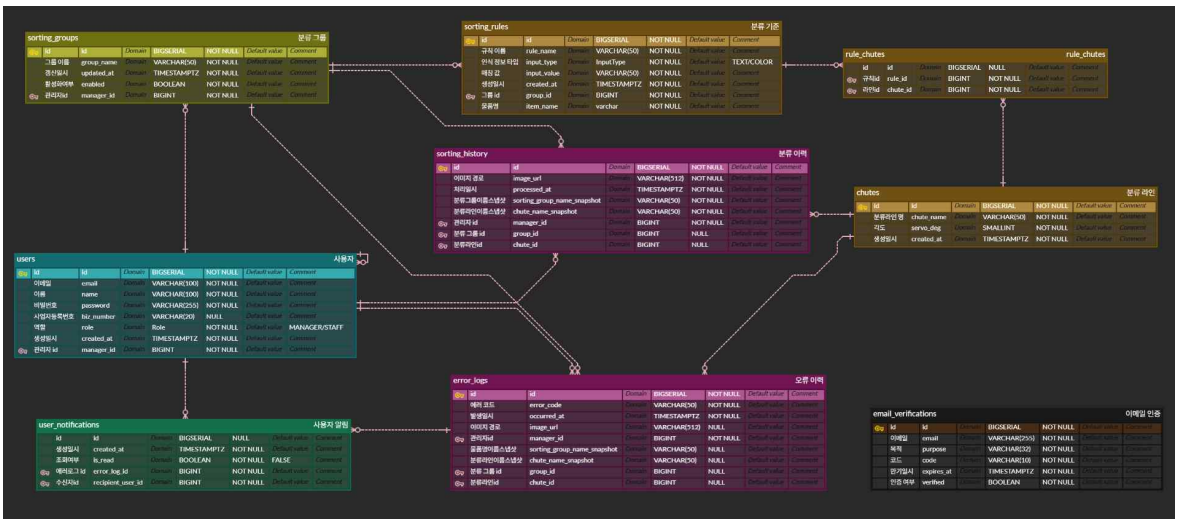
• 성능 개선 여지 및 방안

본 테스트에서 서버는 동시 50명 부하에서도 모든 요청을 100% 성공적으로 처리하며 안정성이 우수하게 확인되었다. 다만, /sorting/history, /stats/*처럼 조회・집계가 많은 API는 부하 시 응답 시간이 증가하는 경향이 있어, 아래와 같은 성능 개선 여지가 존재한다.

1. 캐시(Cache) 도입: 통계성 API는 실시간 집계 쿼리를 매번 수행하므로 부담이 크다. Redis 캐싱을 적용하면 반복 조회의 응답 시간을 크게 줄일 수 있으며, p95 응답 시간을 현재 3~4초 수준에서 1~2초 수준으로 개선 가능할 것으로 생각됨.
2. Azure App Service / DB 플랜 상향: App Service 및 PostgreSQL 상위 티어로 전환 시 동시 요청 처리 성능 향상되어 응답 시간 감소 효과 기대됨.
3. 배치 기반 사전 집계 테이블(Pre-computation) 도입: 일/그룹/슈트 단위 집계 테이블을 갱신하는 구조로 전환하여 조회 시에는 집계 테이블만 읽음으로써, 통계 조회 시 응답 시간을 줄일 수 있을 것이라 생각됨.

# 7. 주요 자료 구조 또는 DB 구조

## 7.1 데이터베이스 설계




본 시스템은 소형 물류 자동화를 위한 분류 및 오류 관리 전 과정을 통합적으로 처리하기 위해 설계되었다. 관리자는 웹/앱에 로그인한 후 분류 그룹을 생성·활성화하고, 각 그룹에 속하는 분류 기준과 라인 매핑을 등록하여 시스템의 자동 분류 로직을 설정한다.

분류 과정에서 디바이스는 촬영된 이미지와 인식 결과를 분류 이력으로 서버에 전송하며, 당시 적용된 그룹명·라인명 등의 정보는 스냅샷 형태로 함께 저장된다. 분류 도중 발생한 오류는 오류 이력으로 기록되며, 오류 코드와 관련 정보가 함께 저장되어 이후 조회·분석에 활용된다.

사용자는 DB에 저장된 권한(관리자/직원)에 따라 분류 그룹 관리, 규칙 설정 등의 기능을 수행할 수 있으며, 이메일 인증을 통해 계정을 안전하게 활성화한다. 또한 시스템은 오류 발생 시 사용자에게 알림을 저장하여 장애 상황이나 분류 실패를 즉시 파악할 수 있도록 지원한다.

서버에 저장된 분류 이력과 오류 이력은 SmartParcel 웹/앱을 통해 직관적으로 확인할 수 있으며, 통계 대시보드에서는 라인별 분류 건수, 일자별 처리 건수, 오류 발생 건수, 전체 처리량 대비 오류 비율 등을 시각화하여 운영자가 실시간으로 시스템의 처리 성능과 안정성을 파악할 수 있도록 제공한다.

## 8. 주요 함수 명세

| 클래스 명 | - |
|---|---|
| 함수 명 | setup |
| 함수 설명 | 아두이노 보드의 초기 상태를 설정하는 함수로, 시리얼 통신을 시작하고 포토인터럽터 입력핀 및 서보모터 핀을 설정한 뒤, 서보모터를 기본(Home) 각도로 이동시켜 시스템을 초기화한다. |
| 인수 | |
| 이름 | 설명 |
| - | 이 함수는 인수를 받지 않는다. |
| 반환 값 | |
| 이름 | 설명 |
| - | 반환 값이 없는 void 함수이다. |
| 소스 코드 | |

```
void setup() {
  Serial.begin(115200);
  delay(2000);
  Serial.println("READY"); // 파이썬 연결 확인용

  pinMode(SENSOR_PIN, INPUT_PULLUP);          // 기존 감지센서
  pinMode(CHECK_SENSOR_PIN, INPUT_PULLUP);  // [추가] 검수센서

  gateServo.attach(SERVO_PIN);
  gateServo.write(HOME_ANGLE);
}
```

| 클래스 명 | - |
|---|---|
| 함수 명 | handleCommand |
| 함수 설명 | 시리얼로부터 한 줄 단위로 수신된 문자열 명령을 파싱하여, PING, HOME, SERVO 명령을 처리하는 함수이다. 각 명령에 따라 서보모터를 기본 각도 또는 지정 각도로 이동시키고, 처리 결과를 시리얼로 응답한다. |
| 인수 | |
| 이름 | 설명 |
| line | 시리얼로 수신한 원본 명령 문자열. 명령어와 각도 인자가 포함될 수 있다. (const String&) |
| 반환 값 | |
| 이름 | 설명 |
| - | 반환 값이 없는 void 함수이다. |

소스 코드

```
void handleCommand(const String& line) {
  int sp = line.indexOf(' ');
  String cmd = (sp == -1) ? line : line.substring(0, sp);
  cmd.trim();
  cmd.toUpperCase();

  if (cmd == "PING") {
    Serial.println("PONG");
    return;
  }

  if (cmd == "HOME") {
    gateServo.write(HOME_ANGLE);
    Serial.print("SERVO_OK ");
    Serial.println(HOME_ANGLE);
    return;
  }

  if (cmd == "SERVO") {
    String arg = (sp == -1) ? "" : line.substring(sp + 1);
    arg.trim();
```

```
        int angle = constrain(arg.toInt(), 0, 180);
        gateServo.write(angle);
        Serial.print("SERVO_OK ");
        Serial.println(angle);
        return;
    }
}
```

| 클래스 명 | - |
|---|---|
| 함수 명 | loop |
| 함수 설명 | 아두이노에서 반복 실행되는 메인 루프로, 두 개의 포토인터럽터(SENSOR_PIN, CHECK_SENSOR_PIN)에 대해 디바운싱을 적용하여 상태 변화를 감지하고, 상태에 따라 “0“, “1“, “CHECK_0“, “CHECK_1“ 문자열을 시리얼로 전송한다. 동시에 시리얼 버퍼를 확인하여 수신된 명령을 한 줄 단위로 누적한 뒤, 개행 문자를 기준으로 handleCommand 함수에 전달해 명령을 처리한다. |
| 인수 | |
| 이름 | 설명 |
| - | 이 함수는 인수를 받지 않는다. |
| 반환 값 | |
| 이름 | 설명 |
| - | 반환 값이 없는 void 함수이다. |
| 소스 코드 | |

```
void loop() {
  unsigned long now = millis();

  // 1) 기존 포토인터럽터 PIN 2 처리
  int cur = digitalRead(SENSOR_PIN);

  if (cur != lastStable) {
    if (now - lastChangeMs >= DEBOUNCE_MS) {
      lastStable = cur;
      lastChangeMs = now;

      if (lastStable == LOW) {
        Serial.println(“0“);    // 기존 센서 LOW 감지
      } else {
        Serial.println(“1“);    // 기존 센서 HIGH 감지
      }
    }
  } else {
    lastChangeMs = now;
```

```
}

 // 2) 검수용 포토인터럽터 PIN 3 처리
int curCheck = digitalRead(CHECK_SENSOR_PIN);

if (curCheck != lastCheckStable) {
  if (now - lastCheckChangeMs >= CHECK_DEBOUNCE_MS) {
    lastCheckStable = curCheck;
    lastCheckChangeMs = now;

    if (lastCheckStable == LOW) {
      // 검수센서 LOW 감지 시 파이썬으로 신호 전달
      Serial.println("CHECK_0");
    } else {
      Serial.println("CHECK_1");
    }
  }
} else {
  lastCheckChangeMs = now;
}

// 3) 파이썬 명령 수신 처리
while (Serial.available()) {
  char c = (char)Serial.read();
  if (c == '\r') continue;
  if (c == '\n') {
    if (rxBuf.length() > 0) {
      handleCommand(rxBuf);
      rxBuf = "";
    }
  } else {
    rxBuf += c;
    if (rxBuf.length() > 64) rxBuf = "";
  }
}

delay(1);
```

```
}
```

| 클래스 명 | DeviceEventService |
|---|---|
| 함수 명 | fetchSetup |
| 함수 설명 | 관리자의 ID를 입력받아, 해당 관리자의 활성화된 분류 그룹을 조회하고, 그룹에 속한 분류 규칙 및 규칙-슈트 매핑 정보를 함께 조회하여 DeviceSetupResponse객체로 구성해 반환하는 함수이다. |
| 인수 | |
| 이름 | 설명 |
| managerId | 설정 정보를 조회할 관리자(User)의 고유 ID 값이다. |
| 반환 값 | |
| 이름 | 설명 |
| DeviceSetupResponse | 조회한 관리자 정보, 활성 분류 그룹, 분류 규칙, 규칙-슈트 매핑 정보를 포함하는 디바이스 초기 설정 응답 DTO이다. |

소스 코드

```
@Transactional(readOnly = true)
public DeviceSetupResponse fetchSetup(Long managerId) {
        User manager = fetchManager(managerId);
        SortingGroup group =
sortingGroupRepository.findFirstByManager_IdAndEnabledTrue(manager.getId())
                .orElseThrow(() -> new NoSuchElementException("활성화된 분류 그룹이 없습니다."));

        List<SortingRule> rules =
sortingRuleRepository.findByGroup_Id(group.getId());
        Map<Long, List<RuleChute>> assignments = rules.isEmpty()
                ? Map.of()
                :
ruleChuteRepository.findByRule_IdIn(rules.stream().map(SortingRule::getId).toList())
                .stream()
                .collect(Collectors.groupingBy(rc -> rc.getRule().getId()));

        return DeviceSetupResponse.from(manager, group, rules, assignments);
}
```

| 클래스 명 | DeviceEventService |
|---|---|
| 함수 명 | recordSortingResult |
| 함수 설명 | 디바이스에서 전송한 분류 결과 요청(DeviceSortingResultRequest)과 촬영 이미지를 입력받아, 관리자・그룹・규칙・슈트가 서로 일치하는지 검증한 뒤 이미지를 스토리지에 저장하고, 이를 기반으로 분류 이력(SortingHistory) 엔티티를 생성하여 DB에 저장하는 함수이다. |
| 인수 | |
| 이름 | 설명 |
| request | 분류 결과 정보(관리자 ID, 규칙 ID, 슈트 ID, 처리 시각 등)를 담고 있는 DeviceSortingResultRequest객체이다. |
| image | 분류 시 촬영된 이미지 파일을 나타내는 MultipartFile객체이다. null이거나 비어 있을 경우 예외가 발생한다. |
| 반환 값 | |
| 이름 | 설명 |
| history.getId() | 새로 저장된 분류 이력(SortingHistory) 레코드의 ID 값이다. |
| 소스 코드 | |

```
@Transactional
    public Long recordSortingResult(DeviceSortingResultRequest request,
MultipartFile image) {
        if (image == null || image.isEmpty()) {
            throw new IllegalArgumentException("이미지 파일이 필요합니다.");
        }

        User manager = fetchManager(request.managerId());
        SortingRule rule = sortingRuleRepository.findById(request.ruleId())
                .orElseThrow(() -> new NoSuchElementException("분류 규칙을
찾을 수 없습니다."));
        SortingGroup group = rule.getGroup();
        if (group == null || !Objects.equals(group.getManager().getId(),
manager.getId())) {
            throw new IllegalArgumentException("요청한 규칙이 해당 관리자에
속하지 않습니다.");
        }

        Chute chute = chuteRepository.findById(request.chuteId())
                .orElseThrow(() -> new NoSuchElementException("슈트를 찾을
```

```
수 없습니다."));
            if (!ruleChuteRepository.existsByRule_IdAndChute_Id(rule.getId(),
chute.getId())) {
                throw new IllegalArgumentException("규칙과 슈트 매핑이 일치하지
않습니다.");
            }

            String storedPath = imageStorageService.store(image, "sorting-results",
manager.getId().toString());

            SortingHistory history = SortingHistory.builder()
                    .imageUrl(storedPath)
                    .processedAt(request.processedAt() != null ? request.processedAt()
: OffsetDateTime.now())
                    .sortingGroupNameSnapshot(rule.getItemName())
                    .chuteNameSnapshot(chute.getChuteName())
                    .manager(manager)
                    .group(group)
                    .chute(chute)
                    .build();

            sortingHistoryRepository.save(history);
            return history.getId();
        }
```

| 클래스 명 | DeviceEventService |
|---|---|
| 함수 명 | recordErrorEvent |
| 함수 설명 | 디바이스에서 전송한 오류 이벤트 요청(DeviceErrorEventRequest)과 선택적인 이미지 파일을 입력받아, 관리자・그룹・규칙・슈트 간의 일관성을 검증한 후 이미지를 스토리지에 저장하고, 오류 코드 및 스냅샷 정보를 포함한 오류 로그(ErrorLog) 엔티티를 생성하여 DB에 기록하는 함수이다. |
| 인수 | |
| 이름 | 설명 |
| request | 오류 코드, 발생 시각, 관리자 ID, 그룹 ID, 규칙 ID, 슈트 ID 등을 포함하는 DeviceErrorEventRequest객체이다. |
| image | 오류 발생 시 촬영된 이미지 파일을 나타내는 MultipartFile객체이다. 필수가 아니며, null또는 비어 있는 경우 이미지를 저장하지 않는다. |
| 반환 값 | |
| 이름 | 설명 |
| errorLog.getId() | 새로 저장된 오류 로그(ErrorLog) 레코드의 ID 값이다. |
| 소스 코드 | |

```
  @Transactional
      public Long recordErrorEvent(DeviceErrorEventRequest request, MultipartFile image) {
          User manager = fetchManager(request.managerId());

          SortingGroup group = null;
          if (request.groupId() != null) {
              group = sortingGroupRepository.findByIdAndManager_Id(request.groupId(), manager.getId())
                      .orElseThrow(() -> new IllegalArgumentException("요청한 그룹을 찾을 수 없습니다."));
          }

          SortingRule rule = null;
          if (request.ruleId() != null) {
              rule = sortingRuleRepository.findById(request.ruleId())
                      .orElseThrow(() -> new NoSuchElementException("분류 규칙을 찾을 수 없습니다."));
              if (rule.getGroup() == null ||
```

```
!Objects.equals(rule.getGroup().getManager().getId(), manager.getId())) {
                    throw new IllegalArgumentException("요청한 규칙이 해당 관리자에 속하지 않습니다.");
                }
                if (group == null) {
                    group = rule.getGroup();
                } else if (!Objects.equals(group.getId(), rule.getGroup().getId())) {
                    throw new IllegalArgumentException("그룹과 규칙 정보가 일치하지 않습니다.");
                }
            }

            Chute chute = null;
            if (request.chuteId() != null) {
                chute = chuteRepository.findById(request.chuteId())
                        .orElseThrow(() -> new NoSuchElementException("슈트를 찾을 수 없습니다."));
                if (rule != null && !ruleChuteRepository.existsByRule_IdAndChute_Id(rule.getId(), chute.getId())) {
                    throw new IllegalArgumentException("규칙과 슈트 매핑이 일치하지 않습니다.");
                }
            }

            String storedPath = null;
            if (image != null && !image.isEmpty()) {
                storedPath = imageStorageService.store(image, "error-events", manager.getId().toString());
            }

            ErrorLog errorLog = ErrorLog.builder()
                    .errorCode(request.errorCode().trim())
                    .occurredAt(request.occurredAt() != null ? request.occurredAt() : OffsetDateTime.now())
                    .imageUrl(storedPath)
                    .sortingGroupNameSnapshot(rule != null ? rule.getItemName() : (group != null ? group.getGroupName() : null))
                    .chuteNameSnapshot(chute != null ? chute.getChuteName() : null)
                    .manager(manager)
                    .group(group)
                    .chute(chute)
                    .build();
```

```
        errorLogRepository.save(errorLog);
        notificationService.notifyError(errorLog);
        return  errorLog.getId();
    }
```

| 클래스 명 | StatsService |
|---|---|
| 함수 명 | countByChute |
| 함수 설명 | 관리자의 이메일과 조회 기간, 선택적인 분류 그룹 ID를 입력받아, 해당 조건에 해당하는 분류 이력(SortingHistory)을 슈트(Chute)별로 집계하여 CountResponse리스트로 반환하는 함수이다. |
| 인수 | |
| 이름 | 설명 |
| email | 집계 대상 관리자를 식별하기 위한 이메일 주소이다. |
| from | 집계 시작 시각. null인 경우 기본 시작 시각으로 보정된다. |
| to | 집계 종료 시각. null인 경우 기본 종료 시각으로 보정된다. |
| groupId | 특정 분류 그룹에 한정하여 집계하기 위한 그룹 ID이다. null이면 전체 그룹을 대상으로 한다. |
| 반환 값 | |
| 이름 | 설명 |
| List<CountResponse> | 슈트 이름과 해당 슈트로 분류된 총 건수를 담은 CountResponse객체들의 리스트이다. |
| 소스 코드 | |

```
@Transactional(readOnly = true)
    public List<CountResponse> countByChute(String email,
                                            OffsetDateTime from,
                                            OffsetDateTime to,
                                            Long groupId) {
        var ctx = sortingContextService.resolve(email);
        Range range = normalizeRange(from, to);
        List<LineCountView> counts =
sortingHistoryRepository.lineCountsByManagerAndDateRange(
                ctx.manager().getId(),
                groupId,
                range.from(),
                range.to()
        );
        return counts.stream()
                .map(view -> CountResponse.of(view.getChuteName(),
view.getTotal()))
                .toList();
    }
```

| 클래스 명 | StatsService |
|---|---|
| 함수 명 | dailyCounts |
| 함수 설명 | 관리자의 이메일과 조회 기간, 선택적인 분류 그룹 ID를 입력받아, 해당 조건에 해당하는 분류 이력을 날짜(일 단위)별로 집계하여 DailyCountResponse리스트로 반환하는 함수이다. |
| 인수 | |
| 이름 | 설명 |
| email | 집계 대상 관리자를 식별하기 위한 이메일 주소이다. |
| from | 집계 시작 시각. null인 경우 기본 시작 시각으로 보정된다. |
| to | 집계 종료 시각. null인 경우 기본 종료 시각으로 보정된다. |
| groupId | 특정 분류 그룹에 한정하여 집계하기 위한 그룹 ID이다. null이면 전체 그룹을 대상으로 한다. |
| 반환 값 | |
| 이름 | 설명 |
| List<DailyCountResponse> | 각 날짜와 그 날 처리된 총 건수를 담은 DailyCountResponse객체들의 리스트이다. |
| 소스 코드 | |

```
  @Transactional(readOnly = true)
      public List<DailyCountResponse> dailyCounts(String email,
                                                  OffsetDateTime from,
                                                  OffsetDateTime to,
                                                  Long groupId) {
          var ctx = sortingContextService.resolve(email);
          Range range = normalizeRange(from, to);
          List<DailyCountView> views =
sortingHistoryRepository.dailyCountsByManagerAndDateRange(
                  ctx.manager().getId(),
                  groupId,
                  range.from(),
                  range.to()
          );
          return views.stream()
                  .map(view -> DailyCountResponse.of(view.getDay(),
view.getTotal()))
                  .toList();
      }
```

| 클래스 명 | StatsService |
|---|---|
| 함수 명 | countByErrorCode |
| 함수 설명 | 관리자의 이메일과 조회 기간, 선택적인 분류 그룹 ID를 입력받아, 오류 로그(ErrorLog)를 오류 코드별로 집계한 결과를 CountResponse리스트로 반환하는 함수이다. |
| 인수 | |
| 이름 | 설명 |
| email | 집계 대상 관리자를 식별하기 위한 이메일 주소이다. |
| from | 집계 시작 시각. null인 경우 기본 시작 시각으로 보정된다. |
| to | 집계 종료 시각. null인 경우 기본 종료 시각으로 보정된다. |
| groupId | 특정 분류 그룹에 한정하여 집계하기 위한 그룹 ID이다. null이면 전체 그룹을 대상으로 한다. |
| 반환 값 | |
| 이름 | 설명 |
| List<CountResponse> | 오류 코드와 해당 코드로 기록된 총 오류 건수를 담은 CountResponse객체들의 리스트이다. |
| 소스 코드 | |

```
@Transactional(readOnly = true)
    public List<CountResponse> countByErrorCode(String email,
                                                OffsetDateTime from,
                                                OffsetDateTime to,
                                                Long groupId) {
        var ctx = sortingContextService.resolve(email);
        Range range = normalizeRange(from, to);
        return errorLogRepository.errorCountsByCodeAndDateRange(
                ctx.manager().getId(),
                groupId,
                range.from(),
                range.to()
        ).stream().map(view -> CountResponse.of(view.getErrorCode(),
view.getTotal()))
                .toList();
    }
```

| 클래스 명 | StatsService |
|---|---|
| 함수 명 | errorRate |
| 함수 설명 | 관리자의 이메일과 조회 기간, 선택적인 분류 그룹 ID를 입력받아, 동일 조건의 총 처리 건수와 총 오류 건수를 각각 조회한 뒤, 오류율을 계산하여 ErrorRateResponse로 반환하는 함수이다. |
| 인수 | |
| 이름 | 설명 |
| email | 집계 대상 관리자를 식별하기 위한 이메일 주소이다. |
| from | 집계 시작 시각. null인 경우 기본 시작 시각으로 보정된다. |
| to | 집계 종료 시각. null인 경우 기본 종료 시각으로 보정된다. |
| groupId | 특정 분류 그룹에 한정하여 집계하기 위한 그룹 ID이다. null이면 전체 그룹을 대상으로 한다. |
| 반환 값 | |
| 이름 | 설명 |
| ErrorRateResponse | 조회 기간 동안의 총 처리 건수, 총 오류 건수, 계산된 오류율(%)을 포함하는 응답 DTO이다. |

소스 코드

```
@Transactional(readOnly = true)
    public ErrorRateResponse errorRate(String email,
                                       OffsetDateTime from,
                                       OffsetDateTime to,
                                       Long groupId) {
        var ctx = sortingContextService.resolve(email);
        Range range = normalizeRange(from, to);

        Long totalProcessed =
sortingHistoryRepository.totalProcessedByManagerAndDateRange(
                ctx.manager().getId(),
                groupId,
                range.from(),
                range.to()
        );
        Long totalErrors =
errorLogRepository.totalErrorsByManagerAndDateRange(
                ctx.manager().getId(),
                groupId,
                range.from(),
                range.to()
```

```
        );
        long processed = totalProcessed == null ? 0L : totalProcessed;
        long errors = totalErrors == null ? 0L : totalErrors;
        double rate = processed == 0 ? 0.0 : (errors * 100.0) / processed;
        return ErrorRateResponse.of(processed, errors, roundTwoDecimals(rate));
    }
```

| 클래스 명 | - |
|---|---|
| 함수 명 | capture_image_from_camera |
| 함수 설명 | 아두이노 센서 신호 수신 시 카메라에서 사진 연속 촬영 후, 가장 선명도가 높은 한 장 선택하여 지정된 폴더에 저장 |
| 인수 | |
| 이름 | 설명 |
| cap | OpenCV VideoCapture 객체 (카메라 연결) |
| save_folder | 이미지를 저장할 로컬 폴더 경로 |
| 반환 값 | |
| 이름 | 설명 |
| save_path | 저장된 이미지 파일의 전체 경로 |
| None | 촬영 실패 시 |
| 소스 코드 | |

```python
def capture_image_from_camera(cap, save_folder):
    for _ in range(3):
        cap.grab()

    best_frame = None
    best_score = -1

    N = 5
    for _ in range(N):
        ret, frame = cap.read()
        if not ret:
            continue

        score = _sharpness_score(frame)

        if score > best_score:
            best_score = score
            best_frame = frame
        time.sleep(0.005)

    if best_frame is None:
        print("[ERROR] 카메라에서 프레임을 읽지 못했습니다.")

        send_error_event(
            manager_id=MANAGER_ID,
            error_code="CAMERA_TIMEOUT",
            image_path=None,
        )
        return None

    filename = f"capture_{int(time.time())}.jpg"
    save_path = os.path.join(save_folder, filename)
    cv2.imwrite(save_path, best_frame)
    print(f"[CAPTURE] 이미지 저장: {save_path}")
    return save_path
```

| 클래스 명 | - |
|---|---|
| 함수 명 | process_image |
| 함수 설명 | YOLO 모델 추론, 이미지 크롭, OCR/HSV 실행해 텍스트/색상 정보 추출 |
| 인수 | |
| 이름 | 설명 |
| image_path | 분석할 이미지 파일 경로 |
| conf_threshold | YOLO 탐지의 최소 신뢰도 기준값 |
| ocr_good_threshold | OCR 인식률의 최소 기준값 |
| 반환 값 | |
| 이름 | 설명 |
| annotated_image | 탐지 결과 표시된 OpenCV 이미지 객체 |
| extracted_data | 텍스트, 색상, 신뢰도 등 분석 결과 담긴 딕셔너리 |
| 소스 코드 | |

```python
def process_image(image_path, conf_threshold=0.7, ocr_good_threshold=0.7):
    if not os.path.exists(image_path):
        print(f"[ERROR] 이미지 파일을 찾을 수 없습니다: {image_path}")
        return None, None

    original_image = cv2.imread(image_path)
    if original_image is None:
        print(f"[ERROR] cv2가 이미지를 읽지 못했습니다: {image_path}")
        return None, None

    orig_h, orig_w, _ = original_image.shape
    print(f"\n--- Processing Image: {image_path} (Size: {orig_w}x{orig_h}) ---")

    results = model(original_image)
    result = results[0]

    final_data = {
        "text": None,
        "text_conf": 0.0,
        "color": None,
        "object_type": None,
        "low_confidence_skips": [],
        "had_text": False,
        "had_color": False,
    }

    found_good_text = False
```

```
    for box in result.boxes:
        conf = box.conf[0].item()
        cls_index = int(box.cls[0].item())
        class_name = CLASS_NAMES[cls_index]

        nx1, ny1, nx2, ny2 = box.xyxyn[0].tolist()
        x1 = int(nx1 * orig_w)
        y1 = int(ny1 * orig_h)
        x2 = int(nx2 * orig_w)
        y2 = int(ny2 * orig_h)

        if class_name == "marker_text":
            bw = x2 - x1
            bh = y2 - y1
            pad = int(0.2 * max(bw, bh))
            x1 = max(0, x1 - pad)
            y1 = max(0, y1 - pad)
            x2 = min(orig_w, x2 + pad)
            y2 = min(orig_h, y2 + pad)

        cropped_img = original_image[y1:y2, x1:x2]

        if class_name == "marker_text":
            final_data["had_text"] = True

            if found_good_text:
                print("[SKIP] 이미 신뢰도 높은 텍스트를 인식해서 나머지 marker_text는 OCR를 건너뜁니다.")
                continue

            if conf < conf_threshold:
                print(f"[WARN] 'marker_text' confidence is low ({conf * 100:.0f}%), 하지만 OCR를 시도합니다.")
                final_data["low_confidence_skips"].append(
                    f"{class_name}_lowconf ({conf * 100:.0f}%)"
                )
            else:
                print(f"[YOLO] Found 'marker_text' (Conf: {conf * 100:.0f}%). Sending to OCR...")

            temp_crop_path = os.path.join(PROJECT_ROOT, "temp_ocr_image.jpg")
            cv2.imwrite(temp_crop_path, cropped_img)

            extracted_text, ocr_confidence = run_ocr(temp_crop_path, multi_angle=True)
```

```
            if extracted_text:
                final_data["text"] = extracted_text
                final_data["text_conf"] = ocr_confidence
                print(f"[SUCCESS] OCR Result: {final_data['text']} (Conf: {ocr_confidence:.2f})")

                if ocr_confidence >= ocr_good_threshold:
                    found_good_text = True
            else:
                print("[INFO] OCR module ran, but found no text.")

            try:
                os.remove(temp_crop_path)

            except Exception as e:
                print(f"[WARN] Failed to remove temp crop file: {e}")
```

```
        elif class_name == "marker_color":
            final_data["had_color"] = True

            if conf < conf_threshold:
                print(
                    f"[WARN] 'marker_color' confidence is low ({conf * 100:.0f}%), "
                    f"하지만 HSV 분석을 시도합니다."
                )
                final_data["low_confidence_skips"].append(
                    f"{class_name}_lowconf ({conf * 100:.0f}%)"
                )

            else:
                print(f"[YOLO] Found 'marker_color' (Conf: {conf * 100:.0f}%). Sending to HSV")

            detected_color = get_color(cropped_img)
            final_data["color"] = detected_color
            print(f"[SUCCESS] Color Result: {final_data['color']}")

        elif class_name == "objects":
            print(f"[YOLO] Found 'objects' (Conf: {conf * 100:.0f}%)")
            final_data["object_type"] = "object"

    return result.plot(), final_data
```

| 클래스 명 | - |
|---|---|
| 함수 명 | classify_and_act |
| 함수 설명 | YOLO/OCR 결과를 기반으로 서버의 분류 규칙과 대조해 최종 분류 라인 및 서보 각도 결정하고 서버 제어 명령 아두이노에 전달하여 분류 성공/실패 여부를 서버에 보고한다. |
| 인수 | |
| 이름 | 설명 |
| image_path | 원본 이미지 경로 |
| annotated_image | 탐지 결과가 표시된 이미지 객체 |
| extracted_data | process_image의 결과 딕셔너리 |
| text_rules | 서버에서 받은 text 분류 규칙 |
| color_rules | 서버에 받은 color 분류 규칙 |
| ser | 아두이노 시리얼 통신 객체 |
| 반환 값 | |
| 이름 | 설명 |
| is_success | 분류 및 서보 제어 성공 여부 |
| servo_moved | 서보 모터가 움직였는지에 대한 여부 |
| saved_path | 결과 이미지 저장 경로 |
| 소스 코드 | |

```python
def classify_and_act(image_path, annotated_image, extracted_data, text_rules, color_rules, ser):
    detected_text = extracted_data.get("text")
    detected_text_conf = extracted_data.get("text_conf", 0.0)
    detected_color = extracted_data.get("color")
    had_text = extracted_data.get("had_text", False)
    had_color = extracted_data.get("had_color", False)
    low_conf_list = extracted_data.get("low_confidence_skips", [])

    rule_id = None
    chute_id = None
    servo_deg = None
    save_path = None
    is_success = False
    error_code = None
    servo_error = False
    servo_moved = False

    base_name = os.path.basename(image_path)
    save_name = f"{os.path.splitext(base_name)[0]}_result.jpg"

    if detected_text and detected_text_conf >= OCR_CONF_THRESHOLD:
        rule_info = text_rules.get(detected_text)

        if rule_info:
            save_path = os.path.join(OCR_OUTPUT_FOLDER, save_name)
            is_success = True
            print("[ROUTE] 정상 텍스트 결과 폴더로 저장")

            rule_id = rule_info["ruleId"]
            chute_id = rule_info["chuteId"]
            servo_deg = rule_info.get("servoDeg")

        else:
            save_path = os.path.join(ERROR_TEXT_OUTPUT_FOLDER, save_name)

            if detected_text.strip().upper() == "UNKNOWN":
                error_code = "UNKNOWN_ERROR"
                print("[ROUTE] UNKNOWN_ERROR: UNKNOWN 텍스트 라벨 감지")

            else:
                error_code = "OCR_FAIL"
                print("[ROUTE] OCR_FAIL: 분류 규칙에 없는 텍스트 라벨")
```

```python
    elif (not detected_text) and (detected_color in KNOWN_COLORS):
        rule_info = color_rules.get(detected_color)

        if rule_info:
            save_path = os.path.join(COLOR_OUTPUT_FOLDER, save_name)
            is_success = True
            print("[ROUTE] 정상 색상 결과 폴더로 저장")

            rule_id = rule_info["ruleId"]
            chute_id = rule_info["chuteId"]
            servo_deg = rule_info.get("servoDeg")

        else:
            save_path = os.path.join(ERROR_COLOR_OUTPUT_FOLDER, save_name)
            error_code = "UNKNOWN_ERROR"
            print("[ROUTE] UNKNOWN_ERROR: 분류 규칙에 없는 색상 라벨")
```

```
    elif (
        (detected_text and detected_text_conf < OCR_CONF_THRESHOLD)
        or any("marker_text" in s for s in low_conf_list)
    ):
        save_path = os.path.join(ERROR_TEXT_OUTPUT_FOLDER, save_name)
        error_code = "OCR_FAIL"
        print("[ROUTE] OCR_FAIL: OCR 신뢰도 낮음 / marker_text low confidence")

    elif (
        (detected_color and detected_color not in KNOWN_COLORS)
        or any("marker_color" in s for s in low_conf_list)
    ):
        save_path = os.path.join(ERROR_COLOR_OUTPUT_FOLDER, save_name)
        error_code = "UNKNOWN_ERROR"
        print("[ROUTE] UNKNOWN_ERROR: 색상 정보 이상(예상 밖 색상 또는 low confidence)")

    else:
        if had_text and not detected_text:
            save_path = os.path.join(ERROR_TEXT_OUTPUT_FOLDER, save_name)
            error_code = "UNKNOWN_ERROR"
            print("[ROUTE] UNKNOWN_ERROR: marker_text는 있었으나 OCR 결과 없음(빈 라벨 등)")

        elif had_color and not detected_color:
            save_path = os.path.join(ERROR_COLOR_OUTPUT_FOLDER, save_name)
            error_code = "UNKNOWN_ERROR"
            print("[ROUTE] UNKNOWN_ERROR: marker_color는 있었으나 HSV 결과 없음")

        else:
            save_path = os.path.join(ERROR_ROOT, save_name)
            error_code = "NO_MARKER"
            print("[ROUTE] NO_MARKER: YOLO가 표식을 탐지하지 못함 (너무 빠르게 지나간 경우 등)")
```

```
try:
    cv2.imwrite(save_path, annotated_image)

    if is_success:
        print(f"[SUCCESS] 결과 이미지를 '{save_path}'에 저장했습니다.")

        if servo_deg is not None:
            if ser is None:
                print("[SERVO] 포트 미연결: SERVO_ERROR")
                servo_error = True
                send_error_event(
                    manager_id=MANAGER_ID,
                    error_code="SERVO_ERROR",
                    rule_id=None,
                    chute_id=None,
                    image_path=save_path,
                )

            else:
                try:
                    main_deg = servo_deg

                    if servo_deg <= 90:
                        PRE_KICK_WAIT = 2

                        print(f"[SERVO] SIMPLE MOVE --> main={main_deg}°, then HOME after {PRE_KICK_WAIT}s")

                        print(f"[SERVO] --> {main_deg}° (분류 위치 이동)")
                        ser.write(f"SERVO {main_deg}\n".encode("utf-8"))
                        time.sleep(PRE_KICK_WAIT)

                        print("[SERVO] --> 0° (back to HOME)")
                        ser.write(b"SERVO 0\n")
                        time.sleep(0.3)
```

```
                    else:
                        PRE_KICK_WAIT = 6
                        KICK_DELTA = 40
                        KICK_WAIT = 0.25

                        back_deg = max(0, min(180, main_deg - KICK_DELTA))

                        print(
                            f"[SERVO] KICK START --> main={main_deg}°, "
                            f"back={back_deg}°, PRE_WAIT={PRE_KICK_WAIT}s"
                        )

                        print(f"[SERVO] --> {main_deg}° (분류 위치 이동)")
                        ser.write(f"SERVO {main_deg}\n".encode("utf-8"))
                        time.sleep(PRE_KICK_WAIT)

                        print(f"[SERVO] --> {back_deg}° (kick back)")
                        ser.write(f"SERVO {back_deg}\n".encode("utf-8"))
                        time.sleep(KICK_WAIT)

                        print(f"[SERVO] --> {main_deg}° (kick forward)")
                        ser.write(f"SERVO {main_deg}\n".encode("utf-8"))
                        time.sleep(KICK_WAIT)

                    reply = ser.readline().decode("utf-8", errors="ignore").strip()
```

```
                    if reply:
                        print(f"[SERVO] from Arduino: {reply}")

                    servo_moved = True

                except Exception as e:
                    print(f"[SERVO] 명령 전송 중 오류: {e}")
                    servo_error = True
                    send_error_event(
                        manager_id=MANAGER_ID,
                        error_code="SERVO_ERROR",
                        rule_id=rule_id,
                        chute_id=chute_id,
                        image_path=save_path,
                    )

        if rule_id is not None and chute_id is not None and not servo_error:
            send_sorting_result(
                manager_id=MANAGER_ID,
                rule_id=rule_id,
                chute_id=chute_id,
                image_path=save_path,
            )

        else:
            if servo_error and not error_code:
                error_code = "SERVO_ERROR"
            if error_code:
                send_error_event(
                    manager_id=MANAGER_ID,
                    error_code=error_code,
                    rule_id=None,
                    chute_id=None,
                    image_path=save_path,
                )
```

```
        else:
            print(f"[INFO] 예외 항목 이미지를 '{save_path}'에 저장했습니다.")
            if not error_code:
                error_code = "UNKNOWN_ERROR"

            send_error_event(
                manager_id=MANAGER_ID,
                error_code=error_code,
                rule_id=None,
                chute_id=None,
                image_path=save_path,
            )

    except Exception as e:
        print(f"[ERROR] 결과 이미지 저장 실패: {e}")
        send_error_event(
            manager_id=MANAGER_ID,
            error_code="UNKNOWN_ERROR",
            rule_id=None,
            chute_id=None,
            image_path=None,
        )

    return is_success, servo_moved, save_path
```

| 클래스 명 | - |
|---|---|
| 함수 명 | run_ocr |
| 함수 설명 | 전처리 적용해 OCR 모델을 실행하고, 가장 신뢰도 높은 텍스트 결과 하나를 반환 |
| 인수 | |
| 이름 | 설명 |
| image_path | 크롭된 이미지의 임시 저장 경로 |
| multi_angle | 다각도 분석 적용 여부 |
| 반환 값 | |
| 이름 | 설명 |
| best_text | 가장 높은 신뢰도로 인식된 텍스트 |
| best_conf | 인식된 텍스트의 최종 신뢰도 |
| 소스 코드 | |

```python
def run_ocr(image_path, multi_angle=False):
    if not os.path.exists(image_path):
        print(f"[ERROR] OCR image file not found: {image_path}")
        return None, 0.0

    image = cv2.imread(image_path)
    if image is None:
        print(f"[ERROR] cv2가 이미지를 읽지 못했습니다: {image_path}")
        return None, 0.0

    print("[DEBUG-OCR] 1단계: fast pass (angle=0, weak preprocess)")
    best_text, best_conf = _try_ocr_with_angles(
        image_bgr=image,
        angles=[0],
        strong=False
    )

    if not multi_angle:
        return best_text, best_conf

    FAST_GOOD_THRESHOLD = 0.70
    if best_conf >= FAST_GOOD_THRESHOLD:
        print(f"[DEBUG-OCR] 1단계 신뢰도 {best_conf:.2f} ≥ {FAST_GOOD_THRESHOLD:.2f}, "
              f"2단계 strong multi-angle 생략.")
        return best_text, best_conf

    print("[DEBUG-OCR] 신뢰도 낮음 -> 2단계 strong multi-angle 시도")

    angles_second_stage = [-15, 0, 15]

    text2, conf2 = _try_ocr_with_angles(
        image_bgr=image,
        angles=angles_second_stage,
        strong=True
    )

    if conf2 > best_conf:
        best_text, best_conf = text2, conf2

    return best_text, best_conf
```

| 클래스 명 | - |
|---|---|
| 함수 명 | get_color |
| 함수 설명 | HSV 공간 분석과 K-means 클러스터링을 통해 표식의 대표 색상 이름을 판정한다. |
| 인수 | |
| 이름 | 설명 |
| bgr_crop | OpenCV 형식의 크롭된 BGR 이미지 배열 |
| 반환 값 | |
| 이름 | 설명 |
| color_name | 판정된 색상 이름 |
| 소스 코드 | |

```python
def get_color(bgr_crop):
    if bgr_crop is None or bgr_crop.size == 0:
        return "Unknown"

    h, w = bgr_crop.shape[:2]
    border = max(6, int(min(h, w) * 0.06))
    x1, y1 = border, border
    x2, y2 = max(x1+1, w-border), max(y1+1, h-border)
    bgr = bgr_crop[y1:y2, x1:x2]
    bgr = cv2.GaussianBlur(bgr, (3, 3), 0)

    hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
    H, S, V = cv2.split(hsv)
    mask = (S >= 70) & (V >= 50) & (V <= 235)

    if not np.any(mask):
        return "Unknown"

    k = max(1, int(min(hsv.shape[0], hsv.shape[1]) * 0.015))
    kernel = np.ones((k, k), np.uint8)
    mask = cv2.erode(mask.astype(np.uint8) * 255, kernel, iterations=1).astype(bool)

    if not np.any(mask):
        return "Unknown"

    h_deg = (H[mask].astype(np.float32) * 2.0)

    if h_deg.size < 30:
        return _band_name(np.median(h_deg)) if h_deg.size else "Unknown"

    theta = np.deg2rad(h_deg)
    feats = np.stack([np.cos(theta), np.sin(theta)], axis=1)

    rng = np.random.default_rng(0)
    centers = feats[rng.choice(len(feats), size=2, replace=False)]

    for _ in range(10):
        d0 = np.sum((feats - centers[0])**2, axis=1)
        d1 = np.sum((feats - centers[1])**2, axis=1)
        labels = (d1 < d0).astype(np.int32)


        c0 = feats[labels == 0].mean(axis=0) if np.any(labels == 0) else centers[0]
        c1 = feats[labels == 1].mean(axis=0) if np.any(labels == 1) else centers[1]
```

```
        def _nz(v):
            n = np.linalg.norm(v) + 1e-9
            return v / n

        new_centers = np.stack([_nz(c0), _nz(c1)], axis=0)

        if np.allclose(new_centers, centers, atol=1e-3):
            centers = new_centers
            break

        centers = new_centers

    n0, n1 = np.sum(labels == 0), np.sum(labels == 1)
    main_c = centers[0] if n0 >= n1 else centers[1]
    main_deg = (np.rad2deg(np.arctan2(main_c[1], main_c[0])) + 360.0) % 360.0

    if 50 <= main_deg <= 90:
        return "YELLOW"

    if _in_band(main_deg, BANDS["RED"]):
        red_ratio = np.mean((h_deg <= 20) | (h_deg >= 340))

        if red_ratio >= 0.55:
            return "RED"

        return "YELLOW" if 45 <= main_deg <= 60 else "Unknown"

    return _band_name(main_deg)
```

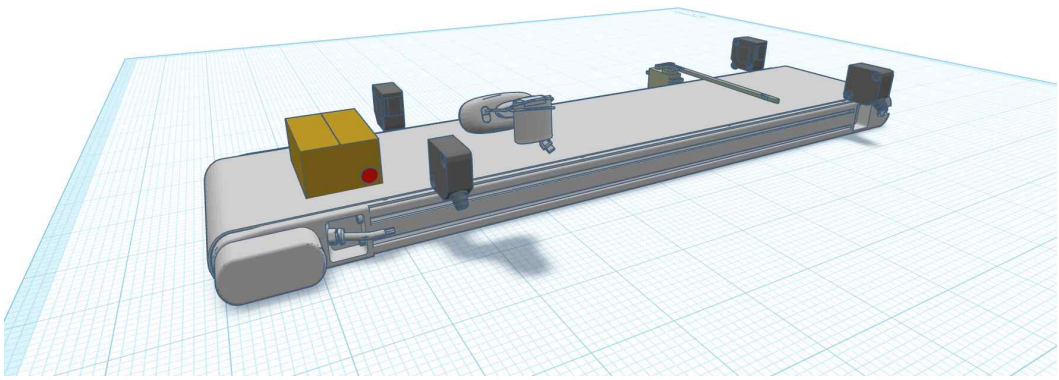
# 9. 설치 매뉴얼 및 사용자 매뉴얼

## 9.1 설치 매뉴얼 (Installation Manual)

### 9.1.1 사전 준비물

| 구분 | 항목 |
|---|---|
| 하드웨어 | Arduino Uno R3 |
| | 컨베이어 벨트 |
| | 투과형 포토센서 2set |
| | 고토크 서보모터 |
| | SMPS 12V/2A(센서용) |
| | SMPS 6V/2A(서보용) |
| | USB Camera |
| 소프트웨어 | Python 3.8 이상 |
| | YOLOv8n ultralytic 8 이상 |
| | PaddleOCR 3.0 이상 |
| | OpenCV 4.0 이상 |
| | Arduino IDE |
| 기타 | 노트북/PC |
| | USB 케이블(아두이노 연결용) |
| | 센서 고정용 브래킷 |
| | 서보 고정용 브래킷 |
| | 삼각대(카메라 고정용) |
| | 점퍼선 |
| | 10KΩ 저항 |

### 9.1.2 시스템 설치 구성도



9.1.3 하드웨어 설치 절차

1) 컨베이어 설치

1. 컨베이어를 최대한 평평한 바닥 위에 고정한다.
2. 전원을 연결하기 전 모터와 롤러가 매끄럽게 회전하는지 확인한다.

2) 포토인터럽터 센서 설치

● 감지용 포토센서

- 분류할 물체 크기를 고려해 컨베이어 프레임 위에 브라켓으로 고정시킨다.
- HIGH/LOW 신호가 감지되도록 송수신기를 광축 정렬한다.

● 검수용 포토센서 (오류 검출 용도)

- 분류 게이트 이후 지점부터 컨베이어 끝부분 사이, 분류에 방해가 되지 않는 지점에 설치한다.
- HIGH/LOW 신호가 감지되도록 송수신기를 광축 정렬한다.

→ 감지용 포토센서는 아두이노 디지털 핀 2번, 검수용 포토센서는 아두이노 디지털 핀 3번에 연결

3) 서보모터 설치

1. 슈트 매핑 시간을 고려해 감지용 포토센서에서 일정 거리 떨어진 지점에서 컨베이어 프레임 위에 브라켓으로 고정시킨다.
2. PWM 신호는 아두이노 디지털 핀 9번에 연결한다.
3. 전원은 6V 2A SMPS를 단독 사용한다. (아두이노 전원과 분리)

4) 카메라 설치

1. USB 카메라를 삼각대에 고정한다.
2. 감지용 포토센서와 거의 동일한 위치에 설치하여 촬영 타이밍을 동기화한다.
3. PC에 USB로 연결한다.

5) 전원 연결

1. 포토센서 → 12V 2A SMPS
2. 서보모터 → 6V 2A SMPS
3. Arduino → USB를 통해 PC 전원 공급
4. 컨베이어 → 전용 AC 전원

9.1.4 소프트웨어 설치 절차

1) Arduino 코드 업로드

1. Arduino IDE 실행
2. 제공된 arduino_sorter.ino 파일 열기
3. 보드: Arduino Uno 선택
4. 포트 선택 후 업로드
5. 업로드 완료 → “READY” 메시지 확인

2) Python 환경 구성

1. Python 3.11 설치
2. 다음 라이브러리 설치

```
pip install ultralytics paddleocr opencv-python pyserial torch numpy requests
```

3. YOLOv8n 모델 파일(yolov8n.pt) 저장
4. PaddleOCR 모델 자동 다운로드 확인

3) AI-하드웨어 연동 실행

```
python main_sorting_service.py
```

- 센서 신호 수신
- 캡처 → 인식 → 규칙 매핑 → 서보 제어 → 검수
- 모든 결과는 서버/DB로 전송됨

4) 웹/모바일 대시보드 연동

- 웹 : 아래 URL에 접속하여 관리자/직원 운영 대시보드 사용 가능.
- https://thankful-bay-05e84e200.3.azurestaticapps.net
- 모바일 : 모바일 앱(.apk)은 추후 웹 대시보드 내에서 직접 다운로드할 수 있도록 제공할 예정.

## 9.2 사용자 매뉴얼 (User Manual)

### 9.2.1 Smart Parcel 시스템 이용 안내문

<Smart Parcel 이용 안내>

원활하고 정확한 자동 분류를 위해 아래 지침을 준수해주시기 바랍니다.

1. 텍스트 표식은 흰색 사각 라벨지를 사용해 주세요.
2. 텍스트 표식의 글자수는 OCR 성능을 위해 최소 2글자, 최대 4글자로 작성해 주세요.
3. 분류 표식은 물체의 오른쪽 하단부에 깔끔하게 부착해 주세요.
(접히거나 주름질 경우 잘 읽지 못할 수 있습니다.)
4. 표식은 매번 비슷한 위치에 붙여주시면 인식 정확도가 더 좋아집니다.
5. 색상 분류 표식은 물체 색과 겹치지 않는 대비되는 색으로 선택해 주세요.
6. 카메라는 포토센서와 거의 동일한 위치에 설치해 주세요.
(포토센서가 인식된 직후 바로 촬영이 이루어집니다.)
7. 시작 전, 카메라 높이를 조정해 표식이 화면에 정확히 들어오도록 맞춰 주세요.
8. 조명이 너무 어두워 표식이 잘 보이지 않을 경우, 휴대용 조명 등으로 표식 부분만 가볍게 밝혀 주세요.
9. 정확한 분류를 위해 물체는 컨베이어 중앙에 맞춰 올려 주세요.

### 9.2.2 장비 조작 절차

1) 시스템 준비

1. 컨베이어 전원 ON
2. 포토센서 2개 정상 동작 확인 (LED 점등)
3. AI 프로그램 실행 → “READY” 상태 확인
4. 카메라 위치 조정 → 표식이 카메라 프레임 안에 들어오도록 맞추기

2) 분류 작업 진행

1. 물체를 컨베이어 중앙에 올린다.
2. 입구 포토센서 감지 → 자동 촬영 → 인식 진행
3. 시스템이 분류 라인에 맞게 서보모터를 동작시킴
4. 배출 후 검수 센서가 최종 통과를 확인
5. 결과는 서버/DB에 자동 저장됨

3) 작업 중 주의사항

- 물체는 일정 간격으로 투입
- 표식이 잘 보이도록 조명 유지
- 컨베이어 위에 이물질, 장애물 두지 않기
- 서보 게이트에 충격 주지 않기

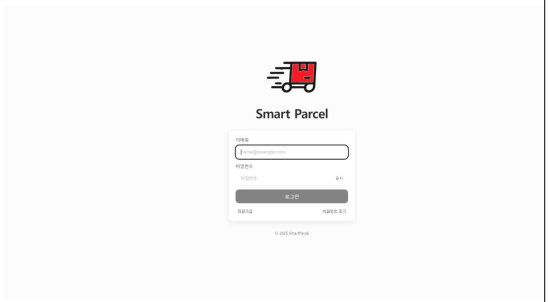
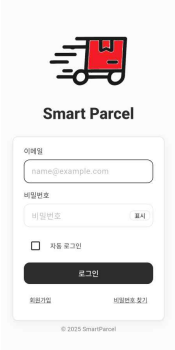
9.2.3 문제 발생 시

시스템 동작 중 예외 상황이 발생하면, 프로그램은 내부에서 send_error_event()를 호출하여 에러코드와 캡쳐 이미지 경로를 서버에 전송한다. 전송된 에러 정보는 관리자 웹의 오류 이력 화면에서 확인할 수 있으며, 각 에러코드는 문제 원인을 빠르게 파악하고 재발 방지 대책을 수립하기 위한 기준으로 사용된다. 본 시스템에서 사용되는 주요 에러코드는 다음과 같다.

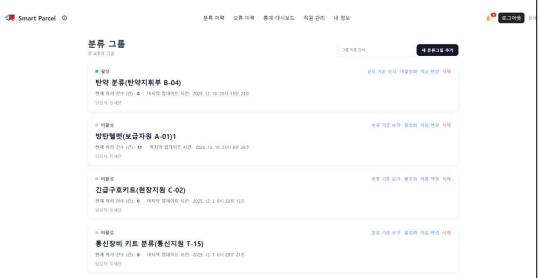
| 에러코드 | 발생 상황 | 상세 설명/조치 |
|---|---|---|
| CAMERA_TIMEOUT | 카메라 프레임 미수신 | • 카메라에서 일정 시간 동안 유효한 프레임을 읽지 못한 경우 발생한다.<br>• 카메라 연결 상태(USB 케이블, 포트, 전원 등)를 확인하고, 필요 시 프로그램을 재시작한다. |
| NO_MARKER | 표식 미검출 | • YOLO가 물체는 감지했으나 marker_text/marker_color 표식을 찾지 못한 경우 사용된다.<br>• 라벨 누락 여부, 컨베이어 속도, 카메라 위치/초점 등을 점검한다. |
| OCR_FAIL | 텍스트 인식 실패 | • 텍스트 영역이 탐지되었으나 OCR 신뢰도가 너무 낮거나 인식된 텍스트가 분류 규칙에 등록되어 있지 않은 경우 발생한다.<br>• 라벨 인쇄 상태(번짐, 폰트)와 규칙 테이블에 해당 텍스트가 등록되어 있는지 확인한다. |
| UNKNOWN_ERROR | 미정의 예외 | • 규칙에 없는 색상이 인식되거나, HSV 결과가 비정상으로 나오거나, 결과 이미지 저장 실패 등의 예외 상황에서 사용된다.<br>• 오류 로그와 저장된 에러 이미지를 참고하여 추가 분석이 필요하다. |
| PHOTO_SENSOR_ERROR | 포토센서 신호 이상 | • 뒤 검수 센서가 앞 센서 트리거 없이 먼저 감지되거나, 내부 상태 정보 없이 뒤 센서만 감지되는 등의 경우에 발생한다.<br>• 설치 위치, 벨트 위 물체 흐름을 점검하여 센서가 올바른 순서로 동작되도록 확인한다. |
| SERVO_ERROR | 서보모터 동작 이상 | • 시리얼 포트를 열 수 없거나, 서보 명령 전송 중 예외가 발생한 경우 등 발생한다.<br>• 아두이노 연결 포트, 서보모터 전원 및 배선을 점검하고 재시도한다. |

9.2.4 대시보드 사용자 매뉴얼

1) 사용자 인증

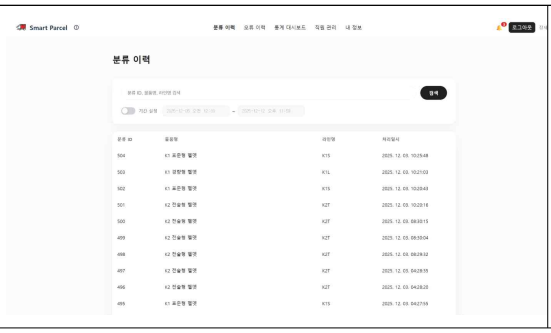
| 웹 | 모바일 |
| --- | --- |
|  |  |
| [회원가입] 이름, 이메일, 비밀번호, 비밀번호 확인을 입력한 뒤, 역할에 따라 관리자는 사업자번호(선택), 직원은 관리자 이메일을 추가로 입력하여 회원가입을 진행한다.<br>로그인 화면에서는 발급받은 이메일과 비밀번호를 입력한 후 [로그인]버튼을 클릭하여 접속한다. 필요한 경우 [자동 로그인] 옵션을 활성화하면 이후 접속 시 인증 절차를 간소화할 수 있다.<br>비밀번호를 분실한 경우 [비밀번호 찾기]메뉴를 통해 재설정할 수 있다. | |

2) 분류 그룹 & 분류 기준

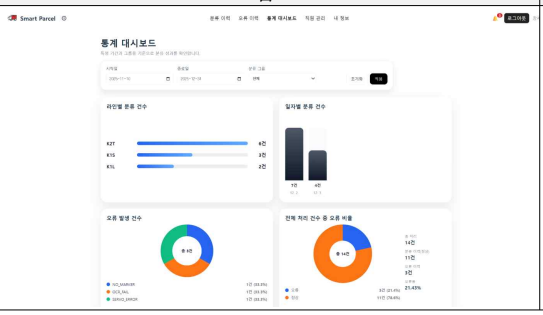
| 웹 | 모바일 |
| --- | --- |
|  |  |
| 대시보드 화면에서는 현재 생성된 분류 그룹 목록을 확인할 수 있다. 각 분류 그룹을 선택하면 해당 그룹에 포함된 분류 기준과 라인 정보를 상세하게 조회할 수 있다. 또한 화면의 [+] 버튼 또는 상단 메뉴를 통해 새 분류 그룹, 분류 기준, 라인을 손쉽게 추가할 수 있다.<br>[관리자]<br>분류 그룹 생성, 활성화/비활성화, 이름 변경, 삭제 기능을 사용할 수 있으며, 필요 시 분류 그룹・분류 기준・라인을 추가하거나 삭제할 수 있다.<br>[직원]<br>분류 그룹 및 기준 정보를 조회만 가능하며, 수정 및 삭제 기능은 제한된다. | |

3) 분류 이력 & 오류 이력

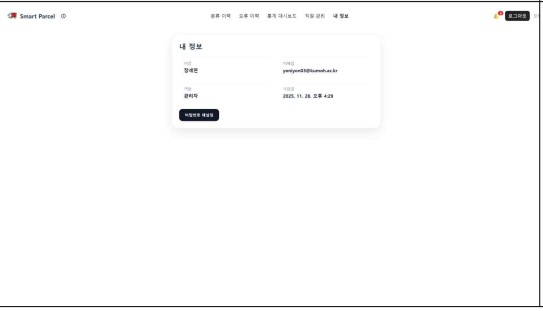
| 웹 | 모바일 |
| --- | --- |

<table>
<tr>
<td>  </td>
<td>  </td>
</tr>
<tr>
<td colspan="2">분류 이력에서는 분류 ID, 물품명, 라인명, 처리일시를 확인할 수 있으며, 오류 이력에서는 오류 ID, 물품명, 라인명, 오류 코드, 처리일시가 함께 표시된다. 각 목록은 검색 및 기간 설정 필터링을 지원하며, 리스트는 페이지네이션 형태로 제공된다.<br>해당 항목을 클릭하면 촬영 이미지가 포함된 상세 정보를 확인할 수 있다.</td>
</tr>
</table>

4) 통계 대시보드

<table>
<tr>
<th>웹</th>
<th>모바일</th>
</tr>
<tr>
<td>  </td>
<td>  </td>
</tr>
<tr>
<td colspan="2">통계 대시보드에서는 라인별 분류 건수, 일자별 분류 건수, 오류 발생 건수, 전체 처리 건수 대비 오류 비율을 그래프와 통계 수치로 확인할 수 있다.<br>또한 시작일 · 종료일, 분류 그룹별로 데이터를 필터링하여 원하는 범위의 통계를 조회할 수 있다</td>
</tr>
</table>

5) 내 정보 & 직원 관리

<table>
<tr>
<th>웹</th>
<th>모바일</th>
</tr>
<tr>
<td>  </td>
<td>  </td>
</tr>
<tr>
<td colspan="2">내 정보 메뉴에서는 사용자의 이름, 이메일, 역할(관리자/직원), 가입일정보를 확인할 수 있으며, 비밀번호 재설정버튼을 통해 비밀번호 변경 화면으로 이동할 수 있다.<br>관리자는 직원 관리 메뉴를 통해 자신의 그룹에 소속된 직원들의 정보를 조회할 수 있으며, 필요 시 해당 직원의 삭제버튼을 통해 관리자-직원 소속 관계를 해제할 수 있다.</td>
</tr>
</table>

6) 알림

| 웹 | 모바일 |
| --- | --- |
|  |  |
| 오류가 발생하면 해당 관리자와 소속 직원에게 알림이 전달된다. 알림 목록에서 각 항목을 클릭하면 상세 내용을 확인할 수 있으며, 동시에 읽음 처리가 된다. 모바일 환경에서는 오류 발생 시 푸시 알림이 자동으로 전송된다. | |

## 10. 역할 분담

| 이름 | 구분 | 담당 역할 |
|---|---|---|
| 김나영 | AI | AI 기반 인식 • 분류 알고리즘 개발 |
| 박진호 | Web/App | 서버 및 사용자 인터페이스 개발 |
| 장세연 | Embedded | 하드웨어 제어 및 센서 연동 |

# 11. 비고

- 박진호

이번 프로젝트는 웹・앱・백엔드・DB・배포 인프라까지 전체 구조를 직접 설계하고 구현한 첫 풀스택 프로젝트였다. 단순히 기능을 만드는 것을 넘어, 하나의 서비스가 실제로 동작하기 위해 필요한 요소들이 얼마나 많고 복잡한지 체감할 수 있는 경험이었다. 특히 인증・권한, 알림, 통계, 분류 규칙/그룹 관리 등 핵심 도메인을 처음부터 정의하고 API를 직접 설계하면서, 기능 하나가 아니라 시스템 전체 흐름을 고려한 설계가 얼마나 중요한지 배우게 되었다. 처음엔 단순 CRUD 중심으로 접근했지만, 점점 사용자 경험과 실제 사용 시나리오를 반영해 구조를 개선하게 되었고, 그 과정에서 도메인 모델링의 중요성을 강하게 느꼈다.

Spring Security 기반 JWT 인증을 구현할 때는 가장 큰 난관을 마주했다. Access/Refresh Token을 적용하면서 보안성과 편의성을 동시에 확보해야 했고, Redis에 Refresh Token JTI를 저장해 재사용 공격을 방지하는 구조를 설계하면서 인증 체계 전반에 대한 깊은 이해가 필요했다. 처음에는 단순히 로그인 후 JWT를 발급하는 정도로 생각했지만, 실제 서비스에서는 토큰 만료, 자동 재발급, 쿠키 정책, Redis TTL, JTI 회전 전략 등 고려해야 할 요소가 매우 많다는 것을 깨달았다.

배포 환경에서도 많은 배움이 있었다. 백엔드는 Azure App Service로 배포하고, 웹 클라이언트는 Azure Static Web Apps를 사용했는데, 특히 Static Web Apps는 GitHub Actions가 자동으로 내장되어 있어 별도의 파이프라인 설정 없이도 자동 빌드・배포가 이뤄졌다. 이번 프로젝트에서는 주어진 워크플로우를 활용하는 수준이었지만, CI/CD가 어떻게 동작하는지 자연스럽게 익힐 수 있었고, 이후에는 직접 커스텀 파이프라인을 구성해보고 싶다는 욕심도 생겼다. 개발자가 동작하는 코드를 넘어서 배포 가능한 코드를 만든다는 것이 무엇인지 실감한 경험이었다.

웹과 앱을 동시에 개발하면서 클라이언트 구조에 대한 고민도 깊어졌다. Flutter와 React는 전혀 다른 환경이지만 동일한 API를 사용해 같은 기능을 구현해야 했기 때문에, 화면 구조와 데이터 흐름이 두 플랫폼 모두에서 일관되게 동작하도록 설계하는 것에 많은 공을 들였다.

또한 분류 이력, 에러 로그, 통계 데이터를 실제로 쌓아보고 조회하는 과정을 통해 데이터 모델링의 중요성도 체감했다. 도메인 설계 과정에서 한 번의 결정이 이후 기능 전체에 영향을 미친다는 것을 몸소 경험할 수 있었고, 사전에 구조를 탄탄히 잡아두는 것이 얼마나 중요한지 알게 되었다.

추가적으로 Redis를 도입해 Refresh Token JTI 저장소로 사용하면서 인메모리 데이터베이스의 장점을 직접 느낄 수 있었다. 현재는 인증 보안을 위한 토큰 관리 용도로만 사용하고 있지만, 실제 서비스에서는 Redis가 캐시 역할로 훨씬 더 다양한 기능을 수행할 수 있다는 점도 확인할 수 있었다. 반복 조회되는 통계・이력 데이터를 캐싱하거나, 실시간 알림을 Redis Pub/Sub 구조로 확장하는 등 성능과 실시간성을 강화할 수 있는 가능성이 많다는 것을 깨달았다. 이러한 경험은 향후 캐시 레이어 설계나 실시간 데이터 처리 구조에 도전해보고 싶다는 동기부여가 되었다.

전체적으로 이번 프로젝트는 단순 기능 구현을 넘어, 백엔드 구조 설계, API 설계, 웹・앱 개발, Redis 기반 인증 아키텍처, Azure 배포까지 전체 개발 라이프사이클을 처음부터 끝까지 경험한 값진 프로젝트였다. 완벽하진 않았지만, 실제 서비스를 지향하는 아키텍처를 직접 만들고 운영해보면서 개발자로서 한 단계 더 성장할 수 있는 계기가 되었고, 다음에는 이번 경험을 기반으로 더 안정적이고 완성도 높은 시스템을 만들어보고 싶다는 목표가 생겼다.

- 김나영

이번 캡스톤디자인은 물체가 규칙에 맞게 정확히 분류될 수 있도록 YOLOv8n, PaddleOCR, HSV 기반 색상 판별 등의 AI 기술을 연동하고, 이를 실제 하드웨어가 동작하도록 구현하는 역할을 담당하게 되었다. 처음에는 단순히 YOLO 모델을 가져와 전이학습을 수행하면 충분할 것이라고 생각했지만, 전이학습을 하기 전 최대한 많은 물체 이미지를 촬영하고 Roboflow에서 전처리를 거친 뒤 전이학습을 해야 했다. 전이학습도 처음부터 좋은 성능의 모델이 만들어지지 않기에 여러 번의 전이학습을 거쳤고 우리 프로젝트에 가장 적합한 모델을 찾는 과정이 쉽지 않아 많은 시행착오를 겪었다.

OCR 인식의 경우, 처음에는 한국어 모델로 시작을 했다. 하지만 생각보다 글씨 크기나 폰트에 민감하게 반응하여 '중형'이라는 텍스트를 '증형'이라고 인식하는 등의 오류가 잦았다. 이후 실제 작업 환경에서는 영어 모델이 더 적합하다고 판단해 모델을 변경했고, 색상 보정이나 회전 기반 multi-angle 전처리 등을 추가한 결과 인식률이 크게 향상되었다.

색상 스티커 분류의 경우에도, 사용하는 카메라가 웹캠이다 보니 환경 특성상 생각보다 조명 변화에 민감했고, 이미지가 흐릿하게 찍히거나 색상 값이 흔들리는 문제가 있었다. 이러한 문제들을 해결하기 위해 Hue 보정이나 원형 K-Means 클러스터링 등을 적용하였으며, 그 후 성능이 크게 개선되어 결과가 잘 나올 때마다 큰 성취감을 느낄 수 있었다. 또한, 수업에서 배웠던 인공지능이나 딥러닝 기술을 실제 프로젝트에서 실용적으로 활용해볼 수 있어 좋았다.

이번 프로젝트를 진행하며 크게 느꼈던 점은 AI 파트와 아두이노 파트의 긴밀한 소통의 중요성이었다. 아두이노 담당 조원과 카메라 촬영, 센서 인식, 분류 각도 제어를 어떻게 연동할지에 대해 매일 논의하고 조정했는데, 실제로 내 코드를 실행했을 때 센서가 정상적으로 작동하고 서보모터가 정확한 각도로 분류할 때 큰 보람을 느꼈다. 각자 역할을 나누어 진행했지만, 각 파트가 서로 어떤 방식으로 구현되고 있는지 이해하는 것이 협업에서 큰 도움이 된다는 점 다시 한 번 느낄 수 있었다.

조금 아쉬웠던 점은 웹캠으로 충분할 것이라고 판단했으나, HSV 색공간을 사용했음에도 초기에는 색상 값이 안정적으로 나오지 않아 많은 시간을 소모했다. 프로젝트에 조금 더 적합한 카메라를 사용했다면 성능도, 개발 속도도 더 좋아졌을 것 같다는 생각이 들었다.

또한, 구매한 서보 모터의 서보 혼이 너무 짧아 3D 프린터로 따로 제작해 길이를 연장했지만 그 마저도 길이가 부족해 분류 동작에 제약이 생겼다. 이 때문에 '킥' 동작을 넣게 된 것이다. 추후에는 서보 혼의 길이를 조금 더 확장하여 별도의 킥 동작 없이도 안정적으로 분류될 수 있도록 개선하면 좋을 것 같다.

프로젝트 초기에는 AI와 아두이노, 웹/앱이 모두 결합된 복합 구조라 정해진 기간 안에 구현이 가능할지, 실제로 AI 관련 프로젝트에 대한 경험도 크게 없어 걱정이 컸다. 그러나 각 팀원들이 맡은 역할을 책임감 있게 수행해준 덕분에 무사히 프로젝트를 완성할 수 있었다. 이번 캡스톤디자인 경험을 통해 팀 프로젝트에서 소통과 협업의 가치를 다시 한 번 실감할 수 있었고, 좋은 팀원들을 만나 프로젝트를 잘 마무리한 것 같아 뜻깊은 시간이었다.

- 장세연

처음으로 우리 아이디어를 직접 시스템으로 구현해보는 프로젝트라 정말 어려움이 많았다. 내가 생각하는 방향을 팀원들에게 정확히 전달하고 공유하는 것부터 쉽지 않았고, 소통이 조금만 어긋나도 바로 문제로 이어졌다. 돌이켜보면 초반엔 서툰 부분도 많았던 것 같다. 그래도 서로 노력하면서 부족한 점을 채워가다 보니 힘들었던 순간들을 하나씩 극복할 수 있었고 좋은 결과도 낼 수 있게 되었다. 그 과정에서 진정한 팀 프로젝트가 무엇인지 경험해 볼 수 있었던 것 같다. 팀원들에게 너무 고맙다.

우선 프로젝트를 시작하기에 앞서 나는 학생이기에 해볼 수 있는 다양한 경험과 도전적인 과제를 수행해보자는 마음가짐으로 임했었다. 만약 실패할지라도 반드시 이런 경험은 미래의 나에게 도움이 될 것이니 최선을 다하자는 생각으로 아이디어 구상을 시작했었는데, 프로젝트가 마무리되는 이 시점에서 생각해보니 궁극적인 목표를 완전히 달성한 것 같아 뿌듯함이 크다.

아이디어를 구체화하는 과정에서는 교수님들께 다양한 피드백을 받으면서 많은 고민이 생겼고, 초반에는 우리만의 차별점을 설득력 있게 잡아가는 데에도 시간이 많이 걸렸다. 때문에 학기 초반에는 혼자 고민도 많이 하고 혼란스러운 부분도 꽤 있었는데, 그 과정에서 팀원들의 솔직한 피드백과 다양한 시각이 큰 힘이 되었고, 프로젝트의 완성도를 끌어올리는 데 중요한 역할을 했다.

임베디드는 이번에 처음 맡은 분야라 특히 막막했다. 관련 강의를 수강해본 경험도 없고 부품에 대한 기본적인 지식도 없어서 하나하나 자료 조사부터 시작해야 했다. 어떤 장비가 우리 시스템 구조와 호환되는지 확인하고, 실제 사용 환경을 고려해 선택하는 데 꽤 많은 시간이 들었다. 국내에서 구하기 어려운 장비도 있었고, 실제로 확인하고 구매할 수 있는 상황이 아니다보니 사전조사와 비교 분석을 충분히 한 뒤에야 결정을 내릴 수 있었다. 포토센서의 경우 종류가 워낙 다양하고 산업용 센서일 시엔 아두이노와 바로 연동이 안되는 경우도 있어 사용 사례를 많이 찾아보고 적용해야 했다. 이외에도 전체 시스템의 흐름을 고려함과 동시에 팀원들의 세부적인 상황들을 체크하며 준비를 했어야했기에 어려움이 많았지만, 그만큼 책임감 있게 임할 수 있었고 이후에 하드웨어적인 부분에서 장비로 인한 큰 문제가 없어 유의미한 시간들이었다고 생각한다.

장비를 모두 마련한 뒤에는 회로를 직접 구성해야 했는데, 유사한 레퍼런스가 거의 없어 외부 전원 연결부터 각 센서 배선까지 하나하나 직접 테스트해야 했다. 그 과정에서 보드가 여러 개 고장나기도 했고, 어느 날은 알 수 없는 유령 신호 때문에 밤새 원인을 추적하며 몇날며칠을 고군분투한 적도 있었다. 배터리의 경우 잔량을 정확히 확인하기 어려워 사용 시간을 대략 계산해 운용했어야 했는데, 나도 모르는 사이에 배터리가 완전히 방전되면 역전류가 흐르며 보드와 주변 장비가 순식간에 손상되는 일이 여러 번 발생했다. 점퍼선을 연결하는 과정에서도 순간적으로 VCC를 GND 단자에 잘못 닿게 해 보드가 타버린 적이 있었는데, 초반에는 이러한 실수 하나가 부품을 완전히 사용불가한 상태로 만들 수 있다는 사실조차 몰라서 고장난 보드를 살려보겠다며 몇 시간을 붙잡고 씨름했던 기억도 있다. 이후 관련 글들과 도서관에서 서적을 빌려 동일한 실수를 반복하지 않기 위한 해결책과 안전한 회로 구성 방법을 스스로 학습해 나갔다. 여러 번의 시행착오 끝에 아두이노와 하드웨어가 제대로 연동되었을 때 느낀 성취감은 정말 컸다. 아직 임베디드 전반을 모두 이해했다고 말할 수는 없지만, 이번 프로젝트를 위해 아두이노와 파이썬 간의 시리얼 통신과 회로, 연동 등 이론적으로 많이 찾아보고 공부했던 과정을 통해 어떤 기술이든 도전하고 문제를 해결해 낼 수 있다는 자신감을 많이 얻게 되었다.

아이디어와 시스템 로직을 팀에 전달하기 위해 간단한 스케치도 많이 해보고, 프론트엔드 디자인과 대시보드 구조를 직접 구성한 부분도 기억에 남는다. 피그마 강의를 찾아보며 툴을 익히고, 화면 설계와 기능 흐름을 구체화하는 과정이 정말 재미있었다. 우리가 생각했던 아이디어가 시각적으로 하나씩 만들어지는 순간들이라 즐거움을 많이 느꼈던 것 같다.

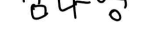
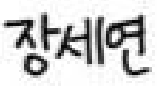
AI 파트에서 YOLO 전이학습 데이터 구축도 도왔는데, 이 작업은 생각보다 훨씬 까다롭고 시간이 오래 걸렸다. 다양한 각도와 위치에서 표식을 붙여 수천 장을 직접 촬영했지만, 작은 차이에도 모델이 과적합되거나 인식률이 떨어지는 문제가 반복됐다. 학습에 걸리는 시간도 길어서 원인 분석 → 재학습 → 성능 비교를 계속 반복해야 했다. 이론 수업으로 배웠던 것과 실제 구현 사이의 간극을 가장 크게 느꼈던 부분이었다. 그래도 끝까지 조정하고 개선한 덕분에 최종 모델은 안정적인 성능을 보여줬고, kit engineering fair와 최종 시연에서 잘 작동하는 모습을 보여드릴 수 있었다.

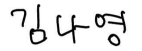
처음 기획하고 이끌어 본 프로젝트라 미숙한 점도 분명있었지만, 그만큼 많이 배운 것 같다. 주제를 직접 찾고, 제한된 기간 안에 설계와 제작, 검증까지 경험해 본 건 정말 값진 시간이었다. 교수님들의 피드백을 즉각 반영하면서 프로젝트를 발전시킬 수 있었던 것도 너무 감사드린다.

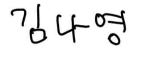
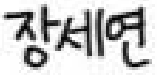
마무리하면서 아쉬움도 있지만, 어떤 부분을 보완해야 하는지 명확히 알게 된 만큼 다음 학기의 캡스톤디자인2에서는 더 나아진 모습으로 이어갈 수 있을 거라 확신한다. 이번 경험은 앞으로 새로운 문제를 마주할 때 큰 기반이 되어줄 것 같다.

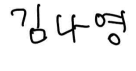
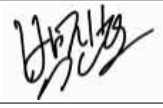
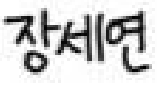
## 12. 회의록

<table>
<tr><td>회의명</td><td colspan="3">캡스톤디자인1 주제 제안서 회의록 1</td></tr>
<tr><td>일 시</td><td>2025년 08월 18일<br>19시</td><td>장 소</td><td>비대면 (Webex)</td></tr>
<tr><td>의 안</td><td colspan="3">역할 분담 및 전체 아키텍처 논의</td></tr>
<tr><td>토의내용</td><td colspan="3">(1) PC-아두이노 통신 구조 필요<br>• YOLO/OCR 결과를 아두이노로 보내 서보모터 각도 제어할 수 있도록 설계<br><br>(2) 하드웨어 구성 보강 필요<br>• 컨베이어 벨트와 슈터 구조 제작<br>• 충분한 토크의 서보모터 필수<br><br>(3) 역할 분담<br>• 컨베이어/아두이노<br>• YOLO + OCR<br>• 웹/모바일</td></tr>
<tr><td>지난 발표 시 지적사항 및 조치 내용</td><td colspan="3">-</td></tr>
<tr><td rowspan="4">참석자 서명 및 불참 사유</td><td>성 명</td><td>서 명</td><td>불참사유</td></tr>
<tr><td>김나영</td><td>김나영</td><td>-</td></tr>
<tr><td>박진호</td><td></td><td>-</td></tr>
<tr><td>장세연</td><td>장세연</td><td>-</td></tr>
<tr><td>작성자</td><td colspan="3">소 속: 컴퓨터공학부 소프트웨어전공 성 명: 김나영 (서명)</td></tr>
</table>

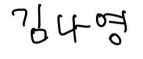
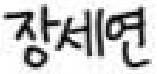
<table>
<tr><td>회의명</td><td colspan="3">캡스톤디자인1 주제 제안서 회의록 2</td></tr>
<tr><td>일 시</td><td>2025년 08월 27일<br>18시</td><td>장 소</td><td>DB134</td></tr>
<tr><td>의 안</td><td colspan="3">데모 시스템 구성</td></tr>
<tr><td>토의내용</td><td colspan="3">(1) 데모 시스템 구성 방향<br>• YOLO로 객체 탐지<br>• OCR로 텍스트 인식<br>• 아두이노가 슈터 제어<br><br>(2) 예산 및 준비 사항<br>• 재료비 필요 → 캡스톤 지원금 신청 예정<br>• 발표 시각적으로 잘 보이도록 구성 필요</td></tr>
<tr><td>지난 발표 시<br>지적사항 및 조치<br>내용</td><td colspan="3">-</td></tr>
<tr><td rowspan="4">참석자 서명 및<br>불참 사유</td><td>성 명</td><td>서 명</td><td>불참사유</td></tr>
<tr><td>김나영</td><td>  </td><td>-</td></tr>
<tr><td>박진호</td><td>  </td><td>-</td></tr>
<tr><td>장세연</td><td>  </td><td>-</td></tr>
<tr><td>작성자</td><td colspan="3">소 속: 컴퓨터공학부 소프트웨어전공 성 명: 김나영 (서명)</td></tr>
</table>

<table>
<tr><td>회의명</td><td colspan="3">캡스톤디자인1 요구 명세서 회의록 1</td></tr>
<tr><td>일 시</td><td>2025년 09월 08일<br>20시</td><td>장 소</td><td>비대면 (Webex)</td></tr>
<tr><td>의 안</td><td colspan="3">프로젝트 방향성 및 데모 구현 보완 논의</td></tr>
<tr><td>토의내용</td><td colspan="3">(1) 프로젝트 필요성 보완<br>• 국방/부대 보급품 등 비정형 물체(수통, 벨트 등) 분류에 저비용 웹캠 + YOLO/OCR 기반 시스템<br>• OCR은 우편 번호가 아닌 커스텀 인식으로 확장<br><br>(2) 최소 구현 범위와 데모 스케일 선정<br>• 실제 물류 센터 수준 구현 어려움<br>• 가능성을 보여주는 수준 + 충분한 크기<br><br>(3) 강조해야 할 핵심 기술<br>• 실시간 학습 어려우므로 사전 학습된 모델 기반으로 데모 구성 → 전이학습 + Roboflow 라벨링<br>• AI 결과가 즉각 물리적 행동으로 이어지는 구조 핵심</td></tr>
<tr><td>지난 발표 시 지적사항 및 조치 내용</td><td colspan="3">(1) 배경 설명 및 필요성 설득이 부족함<br>• 소규모 물류에 대한 자료 조사 및 정리<br><br>(2) 데모 시나리오 개선<br>• 텍스트와 색상 마커로 다양한 분류 인식 방식 가능하도록 수정</td></tr>
<tr><td rowspan="4">참석자 서명 및 불참 사유</td><td>성 명</td><td>서 명</td><td>불참사유</td></tr>
<tr><td>김나영</td><td>  </td><td></td></tr>
<tr><td>박진호</td><td>  </td><td></td></tr>
<tr><td>장세연</td><td>  </td><td></td></tr>
<tr><td>작성자</td><td colspan="3">소 속: 컴퓨터공학부 소프트웨어전공 성 명: 김나영 (서명)</td></tr>
</table>

<table>
<tr><td>회의명</td><td colspan="3">캡스톤디자인1 요구 명세서 회의록 2</td></tr>
<tr><td>일 시</td><td>2025년 09월 10일<br>18시</td><td>장 소</td><td>DB134</td></tr>
<tr><td>의 안</td><td colspan="3">시스템 아키텍처 및 기능 설계 논의</td></tr>
<tr><td>토의내용</td><td colspan="3">(1) 에러 처리 및 로그 구조 설계 논의<br>• 에러 코드 기반으로 오류 유형 구분하는 방식으로 설계 결정<br>• 필요 시 에러 로그 테이블 별도로 분리<br><br>(2) 하드웨어 측 오류와 웹 관리 페이지 오류 구분<br>• 하드웨어 동작 오류 로그와 웹 대시보드에서 관리되는 시스템 오류 이력 구분<br>• 에러 코드 기준으로 분류하는 구조가 적절<br>• 컬럼 추가 또는 테이블 분리 방식<br><br>(3) 기능 리스트 및 유즈케이스 설계 논의<br>• 기능 리스트 분담하여 요구 명세서 작성<br>• 유즈케이스 다이어그램은 직원, 관리자 → 사용자 → 기능 흐름으로 구성</td></tr>
<tr><td>지난 발표 시 지적사항 및 조치 내용</td><td colspan="3">(1) 기능 리스트 및 예외 처리 구성이 미흡<br>• 에러 코드 체계, 기능 분리 기준 수정</td></tr>
<tr><td rowspan="4">참석자 서명 및 불참 사유</td><td>성 명</td><td>서 명</td><td>불참사유</td></tr>
<tr><td>김나영</td><td>  </td><td></td></tr>
<tr><td>박진호</td><td>  </td><td></td></tr>
<tr><td>장세연</td><td>  </td><td></td></tr>
<tr><td>작성자</td><td colspan="3">소 속: 컴퓨터공학부 소프트웨어전공 성 명: 김나영 (서명)</td></tr>
</table>

<table>
<tr><td>회의명</td><td colspan="3">프로젝트 설계 단계 회의</td></tr>
<tr><td>일 시</td><td>2025년 09월 18일<br>19시</td><td>장 소</td><td>DB134</td></tr>
<tr><td>의 안</td><td colspan="3">전체 구조 및 도메인 재설계</td></tr>
<tr><td>토의내용</td><td colspan="3">(1) 도메인 재정립<br>- 교수님 피드백을 기반으로 기존 도메인 재정립 필요. 기존 일반 택배 기반 도메인에서 군수 물자 분류 환경으로 변경함.<br>- 재난구호와 같은 유사 환경에 확장 가능한 도메인.<br>- 저비용, 소형화, 범용성을 갖출 수 있을 것으로 생각됨.<br>(2) 유즈케이스 다이어그램 및 시나리오 수정<br>- 도메인 변경에 따른 유즈케이스 다이어그램 및 시나리오 수정 필요.<br>(3) 분류 로직 구체화<br>- 분류 표식 텍스트, 색상 두 가지로 결정<br>- 텍스트 필기체도 가능한지 논의</td></tr>
<tr><td>지난 발표 시 지적사항 및 조치 내용</td><td colspan="3">(1) 지적사항 : 도메인을 정확하게 잡고 진행해야한다.<br>==> 기존 일반 택배 기반 도메인을 폐기하고, 군수・재난구호 물자 분류 특성을 고려하여 비정형 라벨・현장 대응성・저비용 운영 가능성을 중심으로 도메인을 재정립하기로 함.<br>(2) 지적사항 : 정형화된 라벨보다는 현장에 따라 즉시 변경되는 표식에 대해서 동작하도록 하는 점을 강조해야 할 것 같다.<br>==> 정형화된 라벨에 의존하는 구조가 아닌 색상, 텍스트 두 가지 형태의 표식을 관리자가 직접 설정할 수 있으며 실시간으로 적용 가능한 유연한 구조를 설계하기로 함.</td></tr>
<tr><td rowspan="4">참석자 서명 및 불참 사유</td><td>성 명</td><td>서 명</td><td>불참사유</td></tr>
<tr><td>김나영</td><td>  </td><td></td></tr>
<tr><td>박진호</td><td>  </td><td></td></tr>
<tr><td>장세연</td><td>  </td><td></td></tr>
<tr><td>작성자</td><td colspan="3">소 속 : 컴퓨터소프트웨어공학과 성 명 : 박 진호 (서명)  </td></tr>
</table>

| 회의명 | 프로젝트 설계 단계 회의 | | |
|---|---|---|---|
| 일 시 | 2025년 09월 24일<br>19시 | 장 소 | DB134 |
| 의 안 | 데이터베이스 구조 공유 및 분류 로직 구체화 | | |
| 토의내용 | (1) 데이터베이스 설계 공유<br>- 전체 데이터베이스 구조 공유<br>- 분류 이력, 오류 이력은 과거의 이력으로 당시 물품명, 라인명 등 스냅샷으로 저장됨.<br>(2) 분류 로직 구체화<br>- 분류 표식 필기체 한계, 인쇄체 표식 및 색상 표식 인식률 및 정확도 향상에 집중.<br>- 실제 환경에서 표식 번짐, 조명 변화, 각도 등 다양한 요인 존재, 이에 대응할 수 있는 다양한 환경 학습 데이터 필요.<br>- 텍스트 인식 범위 한글, 영문, 숫자 등 기본적인 문자 유형 처리 가능하도록 설계하여 유연성 확보.<br>(3) API 설계 및 공유 필요<br>- 하드웨어와 서버 간 통신을 명확히 하기위한 API 명세 정리하여 공유 예정. | | |
| 지난 발표 시 지적사항 및 조치 내용 | (1) 지적사항 : 정형화된 라벨 보다는 현장에 따라 즉시 변경되는 표식에 대해서 동작하도록 하는 점을 강조해야 할 것 같다.<br>==> 정형화된 라벨에 의존하는 구조가 아닌 색상, 텍스트 두 가지 형태의 표식을 관리자가 직접 설정할 수 있도록 구조를 유연하게 설계하기로 함. | | |
| 참석자 서명 및 불참 사유 | 성 명 | 서 명 | 불참사유 |
| | 김나영 | 김나영 | |
| | 박진호 | 박진호 | |
| | 장세연 | 장세연 | |
| 작성자 | 소 속 : 컴퓨터소프트웨어공학과 성 명 : 박 진호 (서명) 박진호 | | |

<table>
<tr><td>회의명</td><td colspan="3">프로젝트 개발 단계 회의</td></tr>
<tr><td>일 시</td><td>2025년 10월 22일<br>19시</td><td>장 소</td><td>DB134</td></tr>
<tr><td>의 안</td><td colspan="3">프로젝트 중간 점검 및 공유</td></tr>
<tr><td>토의내용</td><td colspan="3">(1) AI 인식 및 처리<br>- YOLO 기반 객체 인식 모델의 학습 진행 결과를 공유하였으며, 현재 전이학습 3차까지 완료됨. 색상 및 텍스트 인식 정확도가 향상되었으나, 조명・배경・각도 변화에 따른 인식률 저하 문제가 확인됨.<br>- 다양한 조건의 이미지를 추가 수집 및 전이학습 강화를 계획함.<br>(2) 하드웨어 제어<br>- 회로 구성을 완료하여 포토센서와 서보모터의 동작을 확인함. PC와 아두이노 연동을 통해 포토센서 트리거를 받아 카메라가 동작하는 것 확인.<br>- 추후 컨베이어 벨트 환경에서 AI 인식과 제어 타이밍의 동기화가 핵심 과제.<br>(3) 웹/모바일 대시보드<br>- 전체적인 UI와 상당 부분의 REST API 구현됨. 분류 로직을 결정할 분류 기준에 대한 프론트와 백을 완성하였고, 이외의 부가적인 기능들 구현을 추후 진행할 예정.<br>- 하드웨어 연동 및 상호작용을 위한 API 구현을 진행 중이며, 웹이 완성되는 즉시 모바일 개발에 박차를 가할 계획.</td></tr>
<tr><td>지난 발표 시 지적사항 및 조치 내용</td><td colspan="3"></td></tr>
<tr><td rowspan="4">참석자 서명 및 불참 사유</td><td>성 명</td><td>서 명</td><td>불참사유</td></tr>
<tr><td>김나영</td><td>  </td><td></td></tr>
<tr><td>박진호</td><td>  </td><td></td></tr>
<tr><td>장세연</td><td>  </td><td></td></tr>
<tr><td>작성자</td><td colspan="3">소 속 : 컴퓨터소프트웨어공학과 성 명 : 박 진호 (서명)  </td></tr>
</table>

<table>
<tr><td>회의명</td><td colspan="3">프로젝트 개발 단계 회의</td></tr>
<tr><td>일 시</td><td>2025년 11월 10일<br>19시</td><td>장 소</td><td>DB134</td></tr>
<tr><td>의 안</td><td colspan="3">프로젝트 진행 현황 공유</td></tr>
<tr><td>토의내용</td><td colspan="3">(1) AI 인식 및 처리<br>- 컨베이어 벨트 통합 테스트 과정에서 YOLO 기반 객체 인식 전이학습 모델의 과적합 현상이 발생.<br>- 원인은 정형화된 학습 데이터의 과도한 비율로 추정되며, 이에 따라 다양한 조명・흔들림・노이즈 환경의 이미지 데이터를 중심으로 전이학습을 재진행하기로 결정.<br>(2) 하드웨어 제어<br>- 컨베이어 벨트 환경에서 포토센서 구동 및 카메라 촬영 동작이 정상 작동함을 확인하였으며, 타이밍 제어 또한 안정적으로 구현.<br>- API와 연동하여 서보모터 제어 로직을 구현하고, 통합 환경에서의 구동 검증을 수행할 예정.<br>(3) 웹/모바일 대시보드<br>- 하드웨어-서버 연동 관련 API 구현 및 공유.<br>- 웹 대시보드 개발을 마무리 중이며, 이후 모바일 애플리케이션 개발을 즉시 착수할 계획임.<br>- 추후 서버 및 웹 배포 계획.</td></tr>
<tr><td>지난 발표 시<br>지적사항 및 조치<br>내용</td><td colspan="3">(1) 지적사항 : 컨베이어 벨트 환경에서 AI 인식과 제어 타이밍의 동기화가 중요하다.<br>==> AI 인식, 컨베이어 벨트 속도 타이밍 제어 안정적으로 구현 완료.<br>(2) 지적사항 : 컨베이어 벨트가 동작하는 상태에서의 이미지 촬영 및 AI 인식이 실제 환경에 더 적합한 것으로 생각된다..<br>==> 다양한 조명, 흔들림, 노이즈 환경에서 수집한 이미지 데이터를 중심으로 전이학습을 수행하여 인식 정확도를 향상시킬 계획.</td></tr>
<tr><td rowspan="4">참석자 서명 및<br>불참 사유</td><td>성 명</td><td>서 명</td><td>불참사유</td></tr>
<tr><td>김나영</td><td>김나영</td><td></td></tr>
<tr><td>박진호</td><td></td><td></td></tr>
<tr><td>장세연</td><td>장세연</td><td></td></tr>
<tr><td>작성자</td><td colspan="3">소 속 : 컴퓨터소프트웨어공학과 성 명 : 박 진호 (서명)</td></tr>
</table>

# 13. 팀원별 포트폴리오

- 박진호

**1. 역할**

프론트엔드(웹・앱) 및 백엔드 설계・구현・배포. PostgreSQL・Redis 기반 데이터/토큰 관리 구조 설계와 Azure 클라우드 인프라 구축

**2. 기술 스택**

- 개발도구
  IntelliJ IDEA, Visual Studio Code
- 개발언어
  Java 17, JavaScript, Dart
- 프레임워크
  Spring Boot 3.5.4 (Data JPA, Spring Security, Validation, Web, Mail, Redis),
  React.js, Flutter
- DBMS/토큰 저장소
  PostgreSQL, Redis (JWT Refresh Token JTI 저장용)
- 인프라 & 배포
  Microsoft Azure App Service, Azure Static Web Apps,
  Azure Database for PostgreSQL, Azure Cache for Redis

**3. 구현 기능**

- 프론트엔드(웹/앱)
  JWT 기반 인증/계정 관리
  분류 그룹・규칙・슈트(라인) 관리
  분류 이력 및 오류 이력 조회
  통계 대시보드
  알림 기능
  직원/프로필 관리
- 백엔드(API 서버)

• JWT 기반 인증 API(회원가입/로그인/로그아웃/토큰 재발급/이메일 인증/비밀번호 재설정)
  Redis에 Refresh Token JTI 저장하여 토큰 재사용 공격 방지
  Spring Security 기반 MANAGER/STAFF 접근 제어
  그룹・규칙・슈트 CRUD 및 검색/필터 API
  TEXT/COLOR 조건 처리 로직 및 규칙-슈트 매핑 제공
  활성 그룹 정보 및 규칙/슈트 매핑 데이터를 장비에 전달하는 설정 API 제공
  SortingHistory 저장(이미지 포함), 기간/키워드/그룹 기반 조회
  ErrorLog 저장 및 필터링 조회, 상세 정보 제공
  슈트별・일자별・에러 코드별 통계 집계 API
  기간/그룹 기반 전체 처리량 대비 에러율 계산
  에러 발생 시 관리자・STAFF에게 알림 자동 생성
  알림 조회・읽음 처리 API 제공
  SSE 기반 실시간 알림 스트림 제공(웹)
  GlobalExceptionHandler로 Validation/권한/비즈니스 예외 일관 응답

- 김나영

**1. 역할**

AI 기반 객체 인식 및 텍스트 인식 시스템 개발 담당

**2. 기술 스택**

- 개발 도구
  - PyCharm
  - Roboflow
  - Git / GitHub

- 프레임워크
  - YOLOv8n (Ultralytics)
  - PaddleOCR
  - OpenCV

- 개발 언어
  - Python

**3. 구현 기능**

- 데이터 수집 및 라벨링 자동화 환경 구축
  - YOLO 학습을 위해 수백 장의 물체 및 표식 이미지 직접 촬영
  - Roboflow를 활용하여 객체 정의(objects, marker_text, marker_color) 및 라벨링 작업 수행

- YOLOv8n 객체 탐지 모델 전이학습
  - 커스텀 데이터셋 기반 YOLOv8n 모델 학습
  - Confusion Matrix 기반 오인식 패턴 분석
  - 여러 차례의 모델 재학습을 통해 탐지 정확도 최적화

- 텍스트 인식(OCR) 모듈 구현
  - PaddleOCR 기반 텍스트 인식 파이프라인 구현
  - multi-angle OCR, 문자 정규화, 신뢰도 기반 필터링 등 로직 제공

- 색상 기반 분류 기능 구현
  - HSV 컬러 공간 변환
  - 노이즈 감소를 위한 전처리

- AI 통합 분류 파이프라인 구현
  - YOLO 객체 탐지 → 텍스트 인식(OCR) → 색상 판정 라인 결정으로 이어지는 전체 AI 파이프라인 하나의 흐름으로 통합
  - 예외 처리, 신뢰도 임계값 설정 등 시스템 안정성 확보

- 실제 장비(AI → Arduino → Servo) 연동 테스트
  - YOLO + PaddleOCR 결과를 바로 서보모터 제어 신호로 변환
  - 카메라-AI-서보모터가 실시간으로 연동되도록 전체 프로세스 검증

- 장세연

1. 역할
본 프로젝트에서 임베디드 하드웨어 설계·제작·제어 전체를 총괄했으며,
팀장(rise 사업단 주관 캡스톤디자인)으로서 시스템 아키텍처 설계, 기능 정의, 기술 의사결정 및 일정/문서 관리를 주도했다.

- 임베디드 하드웨어 총괄
  - 컨베이어 환경 분석을 기반으로 한 주요 장비(포토센서·서보모터·전원부) 사양 검토 및 선정
  - 하드웨어 구성(센서·서보·카메라)과 전원·배선 구조의 전체 설계
  - 3D 프린터를 활용한 서보혼 자체 설계 및 제작
  - 컨베이어 + 센서 + 카메라 + 서보 게이트를 포함한 시스템 전체 물리 구조 구축

- 임베디드 소프트웨어(Arduino) 개발
  - Arduino 기반 센서·서보 제어 구조 설계
  - Python-Arduino 양방향 통신 구조 정의 및 동작 표준 정립

- AI-하드웨어 연동 파이프라인 구축
  - 센서 감지 → 이미지 캡처 → YOLO/PaddleOCR 기반 인식 → 규칙 매칭 → 서보 제어 → 검수 센서 확인 End-to-End 자동 분류 파이프라인 로직 전체 설계
  - 컨베이어 속도·물체 마찰력·서보 토크를 고려한 분기 타이밍 최적화
  - 통신 명령 구조(HOME/SERVO/PING) 표준화 및 Python/Arduino 간 연동 로직 확립

- AI 데이터 구축 참여
  - YOLOv8 전이학습용 데이터 촬영·라벨링 참여
  - 인식률 향상을 위한 조명·각도·환경 다양성 확보 전략 제안

- UI/UX 디자인
  - 웹·모바일 전체 화면 구조(Figma) 디자인 및 기능 흐름(UX) 설계

- 프로젝트 리딩
  - 주간 회의 운영 및 전체 일정 관리
  - 도메인 재정립(군수·재난 물자) 과정에서 전체 시스템 구조/로직 통합 재설계
  - 계획서·기자재·예산·전시 운영·회의록 등 프로젝트 전체 문서화 관리

2. 기술 스택
- Embedded / Hardware
  Arduino C++, Arduino IDE
  Serial Communication(Python-Arduino), PWM Control
  3D Modeling(서보혼 커스텀 제작), 회로 설계 및 테스트

- Vision 연동
  Python, PySerial, OpenCV
  YOLOv8n, PaddleOCR 기반 인식 파이프라인 설계

- UI/UX
  Figma, Canva - 웹/모바일 UI 구성·프로토타입 제작

3. 구현 기능

1. 임베디드 하드웨어 시스템 구축

- 포토센서 설치 지점・감도・지연 특성 검토 및 최적 배치
- 실제 분기 동작을 위해 서보모터・브라켓・서보혼의 기구 설계-전원-신호부가 연동되는 통합 제어 구조 설계
- 전원 분리 운용 중 발생한 전압 강하・역전류・노이즈 문제를 실측・테스트를 통해 원인 분석 및 개선
- 컨베이어 속도・물체 이동 패턴・센서 지연시간을 고려한 카메라 트리거-인식-서보 제어 타이밍 보정

2. Arduino 제어 로직 개발

- 디바운싱 기반 센서 신호 안정화 로직 구현
- 감지・검수 센서 독립 상태 추적 구조 설계
- Python 수신 명령 처리용 커스텀 프로토콜 구현 및 모터 동작 결과 피드백 설계

3. Python-Arduino 통신 및 시스템 로직 설계

- 시리얼 양방향 통신 패킷 정의
  (“0/1”, “CHECK_0/1”, “SERVO 90”, “HOME”, “PONG” 등)
- AI 인식 결과가 아두이노 제어로 자연스럽게 이어지도록 Decision-Action 파이프라인 설계

4. 분류 로직 설계

- 텍스트/색상 표식 기반 라인 선택 규칙 구조 수립
- 규칙 매칭 및 예외 처리(인식 실패 시 예외 슈트 배출) 설계
- 물체 이동 속도・크기에 따른 분류 게이트 동작 최적화

5. 대시보드 UX/UI 디자인

- 웹・모바일 전체 화면 구조(Figma) 설계 및 모든 주요・세부 화면 디자인
- Rule Mapping, 기준 관리 등 운영 기능을 고려한 사용자 흐름(UX) 정립 및 인터랙션 구성

6. 프로젝트 관리

- 설계・구현 단계별 회의 주도 및 아키텍처 결정
- 도메인 전환 이후 전체 시스템 재정립